
Aprender a Programar em 3 Meses

Do zero ao teu primeiro código em 90 dias

OSENDRÉ PLUS

*Livros que se leem numa semana
e mudam o resto da vida*

Aprender a Programar em 3 Meses

OSENDRÉ PLUS

© 2026 Osendre Plus. Todos os direitos reservados.

1.^a edição.

*Este é um livro original, escrito de raiz. Não foi copiado,
traduzido nem adaptado de ninguém — nasceu aqui, para ti,
para a nossa gente e a nossa terra.*

Nenhuma parte deste livro pode ser reproduzida ou vendida
sem autorização da Osendre Plus.

PREFÁCIO

Os livros da Osendre Plus nascem de uma convicção simples: o conhecimento que muda uma vida não devia estar preso em livros caros, difíceis e escritos para outra gente que não a nossa. Escrevemos para ti — jovem de Angola e dos PALOP — na tua língua, com a tua realidade e com exemplos que reconheces à primeira.

Este livro é curto de propósito, para o leres numa semana, e prático de propósito, para o fazeres e não apenas o leres. No fim de cada capítulo há um exercício, o «Faz isto hoje». É aí que a mudança acontece de verdade — não nas páginas que lês, mas nas coisas que passas a fazer por causa delas. Não os saltes.

Boa leitura. E melhor ainda: boa prática.

— Osendre Plus

ÍNDICE

Introdução	1
Porquê aprender a programar agora	3
Monta o teu ambiente de trabalho	8
Lógica de programação: o pensamento passo a passo	14
Variáveis, tipos e operadores	20
Estruturas de controlo: loops e decisões	25
Funções: reutiliza o teu código	31
Trabalhar com dados: listas e dicionários	37
Introdução ao desenvolvimento web	43
Projeto simples: cria a tua primeira aplicação	48
Depuração e tratamento de erros	54
Controlo de versão com Git	61
Boas práticas e legibilidade	67
Preparar o teu portefólio	73
Plano de 90 dias para tornar-te programador	77

.....

INTRODUÇÃO

O sol ainda lançava os primeiros raios sobre o Mercado da Praça da Independência, quando o candongueiro de Vanda, com a caixa cheia de moedas de 100 kwanzas, parou ao teu lado. A tua mão tremia ao receber o troco, mas o que realmente te deixava inquieto era o ecrã do telemóvel: um vídeo de um jovem a escrever linhas de código que, num instante, transformavam números em gráficos de música. Vendeu-se a ideia de que, com um teclado, podias mudar a tua vida, mas também sentias o peso de quem nunca tinha aberto um editor de texto.

Lembro-me de ter sentado-me naquela mesma banca, a observar o movimento das pessoas, a escutar o barulho dos vendedores a anunciar os seus produtos, e a perguntar-me como poderia, eu também, entrar naquele mundo. Foi então que percebi que aprender a programar não era um privilégio reservado a quem estudou no exterior, mas uma porta que se abre à medida que se conhece as ferramentas certas e se pratica com constância.

Neste livro vais descobrir, passo a passo, como transformar a curiosidade que sentes agora em competência real. Não vais encontrar teorias vazias nem promessas de milagres; vais encontrar um caminho claro, dividido em três meses que, se seguidos, te levarão do zero ao primeiro programa funcional. Vais aprender a escolher a linguagem que melhor se adapta aos teus objetivos, a configurar o ambiente de desenvolvimento no teu computador ou no teu telemóvel, e a escrever código que realmente funciona, testando-o e melhorando-o.

Ao longo das páginas, vais encontrar exemplos tirados da vida quotidiana – calcular o preço de uma compra no mercado, automatizar a gestão das tuas poupanças, criar um pequeno jogo que pode ser partilhado com os amigos. Cada capítulo traz exercícios práticos, dicas para ultrapassar bloqueios e sugestões para continuar a evoluir depois de concluir o terceiro mês.

Se agora sentes que a programação pode ser a chave para abrir novas oportunidades, seja no teu bairro, na tua escola ou no futuro trabalho que ainda não consegues imaginar, este livro é o teu primeiro passo. Vai ser um investimento de tempo que vale cada kwanza que decides gastar. Vira a página.

CAPÍTULO 1

Porquê aprender a programar agora

Amanhã, quando o candongueiro já tinha largado o cheiro de gasolina no ar, avistei o Zé Miguel a montar uma bancada improvisada na esquina da Rua da Boa Esperança, em frente à escola secundária de Luanda. Ele tinha um velho portátil, um teclado que rangia a cada pressionar, e um caderno onde anotava, com caneta azul, linhas de código que ainda não tinha conseguido fazer funcionar. “Olha, tio, ainda não corre”, disse-me ele, com os olhos a brilhar de frustração, enquanto a senhora da padaria lhe oferecia um copo de água de coco. Aquilo não era só um momento de curiosidade; era um ponto de partida. Se ele, que ainda está a terminar o ensino secundário, consegue transformar ideias em programas, tu também podes, mesmo que o teu ponto de partida seja um telemóvel antigo ou um computador partilhado na casa da avó.

O código como moeda da nova economia

A realidade que vemos no mercado de trabalho mudou de forma tão rápida que as oportunidades de quem domina a programação já não são exclusividade de quem estudou em universidades de elite. Hoje, uma aplicação que ajude a organizar os pagamentos de um candongueiro, ou um site que mostre os preços dos peixes no mercado de Mussulo, pode gerar renda suficiente para sustentar uma família. Empresas de tecnologia, startups locais e até projectos de ajuda humanitária procuram alguém que saiba transformar um problema

em algoritmo. Não precisas de um diploma de Engenharia Informática para ser valorizado; precisas de capacidade de criar, de entender a lógica que faz os sistemas funcionarem.

Num país onde a taxa de desemprego ainda pesa sobre muitos jovens, a programação abre uma porta que se abre para o futuro, mas também para o presente. Muitos dos programadores que hoje trabalham em projetos de energia renovável ou de educação a distância começaram por criar pequenos programas para resolver problemas da vida quotidiana. O que antes era um “hobby” tornou-se uma fonte de renda constante. O facto de seres capaz de construir algo do zero – um site, uma app, um script que automatiza tarefas – coloca-te numa posição de negociação que poucos outros têm.

O impacto na vida quotidiana

Quando te perguntas por que aprender a programar, pensa nas tuas próprias rotinas. Já alguma vez tiveste de organizar as tuas notas da escola, gerir as finanças da tua família ou planear o teu percurso académico? Imagina um pequeno programa que, a partir de um ficheiro Excel, calcula automaticamente quanto cada um deve pagar ao final do mês, alertando-te quando o orçamento está a ficar apertado. Ou então, uma aplicação que te avisa quando há um buraco na estrada que o teu candongueiro costuma usar, com base em relatos de outros motoristas.

A programação permite-te criar ferramentas que te poupam tempo, evitam erros e melhoram a qualidade da tua vida. Não é só sobre ganhar dinheiro; é sobre ter controlo. Quando o teu amigo começou a aprender Python e escreveu um script que lhe enviava um e-mail toda vez que a conta da eletricidade ultrapassava um certo limite, ele passou a ter clareza sobre o consumo e a conseguir negociar melhores tarifas. Esse mesmo princípio pode ser aplicado a qualquer área:

agricultura, comércio, saúde. O código torna-se uma extensão da tua capacidade de decisão.

A aprendizagem é mais acessível do que nunca

Talvez te preocupe não teres acesso a recursos caros ou a um professor particular. A verdade é que, em 2024, há milhares de recursos gratuitos em português, desde tutoriais em vídeo até comunidades de desenvolvedores que falam a nossa língua. O próprio Zé Miguel aprendeu a programar usando vídeos no YouTube e fóruns onde outros jovens partilhavam dúvidas. Não precisas de um laboratório de informática de última geração; um portátil com capacidade mínima já te permite praticar. O que realmente conta é a disciplina de reservar alguns minutos todos os dias para escrever código, experimentar e corrigir erros.

Além disso, o conceito de “programar” não é limitado a uma única linguagem. Começar por Python, por exemplo, permite-te aprender os fundamentos – variáveis, estruturas de controlo, funções – de forma intuitiva. Depois, poderás transitar para outras linguagens, como JavaScript para desenvolvimento web, ou até mesmo para linguagens mais específicas de ciência de dados. O ponto de partida é simples: escrever linhas de código que façam algo concreto. Cada pequeno sucesso alimenta a confiança e cria um efeito dominó que te leva a projetos maiores.

Programar como forma de pensamento crítico

Aprender a programar vai além da escrita de código; desenvolve um modo de pensar que é valioso em qualquer contexto. Quando enfrentas um bug, precisas de analisar o problema, dividir-o em partes menores, testar hipóteses e encontrar a solução. Esse processo de resolução de problemas, de decomposição e de iteração, é

exatamente o que os empregadores valorizam. Não é surpresa que muitas empresas procurem candidatos que demonstrem “pensamento algorítmico”, mesmo que a posição não seja de desenvolvimento puro.

Ao aplicar esse raciocínio a situações do dia a dia – como organizar um evento, planejar uma viagem ou melhorar um processo no trabalho – vais perceber que a programação oferece um modelo mental para lidar com a complexidade. Cada desafio torna-se um conjunto de passos lógicos que podem ser seguidos, testados e refinados. Essa capacidade de transformar um problema nebuloso num conjunto de instruções claras é a base para o sucesso em qualquer área.

O futuro que já está a acontecer

Olha ao teu redor: as escolas estão a introduzir aulas de informática; as empresas de telecomunicações lançam programas de formação em desenvolvimento de software; as organizações não-governamentais criam plataformas digitais para melhorar a transparência dos projetos. Se te juntas a este movimento agora, estás a posicionar-te na vanguarda de uma transformação que ainda vai acelerar. A tecnologia não espera; ela avança, e quem tem as competências certas pode escolher o rumo da própria história.

A decisão de aprender a programar não tem de ser um salto gigantesco. Cada linha que escreves, cada pequeno projecto que concluis, acrescenta-te ao grupo de jovens que, com as mãos na tecnologia, constroem as soluções de amanhã. Tu tens a oportunidade de transformar o teu futuro e, ao mesmo tempo, contribuir para o desenvolvimento da tua comunidade. Não é só uma questão de “porquê”; é uma questão de “quando”. O momento é agora, e a porta está aberta àqueles que têm a coragem de entrar.

FAZ ISTO HOJE

Instala Python no teu computador ou usa um ambiente online como Replit.

Cria um ficheiro chamado ``despesa.py``.

Escreve um programa que peça ao utilizador o salário mensal e o valor das despesas fixas (ex.: água, luz, internet) e calcule quanto sobra.

Executa o programa com diferentes valores para perceberes como pequenas variações afetam o resultado.

Anota no teu caderno o que aprendeste e identifica um outro problema da tua vida quotidiana que poderias resolver com um pequeno script.

CAPÍTULO 2

Monta o teu ambiente de trabalho

Na esquina da Rua 1 de Junho com a Avenida 4 de Fevereiro, o candongueiro de Maria do Carmo já está a fazer a ronda matinal. Enquanto ele vai de casa em casa, tu estás sentado numa cadeira de plástico, à frente do teu portátil que acabou de chegar ao teu quarto no bairro da Terra Nova, ainda com o papel de embrulho ainda colado na parte de trás. O som da música de Kizomba que sai da loja de discos ao fundo mistura-se ao cheiro de feijão com óleo de palma que vem da cozinha da tua irmã. Tens 20kwanzas na carteira, um pen drive velho que o teu primo usou para guardar fotos de família, e a vontade de transformar aquele ecrã em algo que te pague a conta de luz. Este é o ponto de partida: o teu ambiente de trabalho ainda não existe, mas está prestes a nascer.

Escolhe o teu hardware com consciência

Não precisas de um computador de última geração para começar a programar. O que realmente conta é a estabilidade e a capacidade de correr os programas que vais escrever. Se tens um portátil que já consegues usar para navegar na internet, abrir documentos e assistir a vídeos, esse já serve. Verifica que o processador tem, no mínimo, duas núcleos e que a memória RAM está acima de 4GB; isso garante que o editor de código e o terminal não ficarão presos.

Se ainda não tens um portátil, pensa em comprar um usado. Nos mercados de segunda mão de Luanda, é comum encontrar máquinas com boas condições por menos de 30 000 kwanzas. Antes de fechar negócio, liga-te a uma tomada, abre o navegador e verifica se o ecrã responde bem ao toque, se o teclado tem todas as teclas e se a bateria ainda tem carga suficiente. Não precisas de um SSD, mas se o encontrarem a um preço razoável, é um investimento que acelera a abertura de projetos.

A monitorização do ambiente físico também conta. Um local bem iluminado, uma cadeira confortável e um apoio para o portátil – uma caixa de papelão ou um suporte improvisado – evitam dores no pescoço e no dorso depois de horas a codificar. Não subestimes o impacto de um ambiente que te faça sentir bem; a produtividade nasce da combinação entre mente e corpo em equilíbrio.

Instala o software essencial

Depois de garantir o hardware, chega a hora de colocar no ecrã as ferramentas que vais usar todos os dias. Começa por instalar um sistema operativo que te dê controlo total. Se o teu portátil já vem com Windows, podes mantê-lo, mas há uma forte comunidade que recomenda Linux, porque oferece rapidez, segurança e acesso a milhares de pacotes de desenvolvimento. Distribuições como Ubuntu ou Mint são fáceis de instalar e têm suporte em português, o que facilita a aprendizagem.

Para instalar Linux, baixa a imagem ISO do site oficial, grava-a num pen drive usando uma ferramenta como Rufus (disponível para Windows) e arranca a máquina a partir do pen drive. Segue as instruções de instalação, escolhendo a partição que preferires. Se preferires não mudar o sistema operativo, podes usar o Windows Subsystem for Linux (WSL) para correr um ambiente Linux dentro do

próprio Windows – basta habilitar a funcionalidade nas Configurações, instalar a distribuição desejada e já tens acesso ao terminal Bash.

Com o sistema operativo pronto, instala o Git. O Git é a ferramenta que vai permitir que guardes o teu código, trabalhes em equipa e voltes atrás se algo correr mal. No Linux, basta abrir o terminal e correr ``sudo apt install git``. No Windows, descarrega o instalador no site oficial e segue as instruções. Depois de instalar, configura o teu nome e e-mail com:

```
``` git config --global user.name "O teu Nome" git config --global user.email "teu@email.com" ```
```

O próximo passo é escolher um editor de código. Existem duas abordagens principais: editores leves como Visual Studio Code (VSCode) ou Sublime Text, e IDEs completas como PyCharm ou IntelliJ. Para quem está a começar, VSCode oferece um bom equilíbrio entre simplicidade e extensibilidade. Descarrega-o do site da Microsoft, instala-o e abre-o. Dentro do VSCode, vai à aba de extensões e procura por “Python”, “JavaScript” ou a linguagem que preferires; instala-as para teres realce de sintaxe, autocompletar e linting (aviso de erros).

Não te esqueças de instalar a linguagem de programação que vais usar. Se vais iniciar com Python, corre ``sudo apt install python3`` no Linux ou descarrega o instalador do site oficial para Windows. Para JavaScript, o Node.js traz o interpretador e o gestor de pacotes npm; instala-o a partir do site oficial. Estas instalações criam as bases para correr os teus scripts e instalar bibliotecas adicionais.

## **Configura o teu editor para ser teu aliado**

Um editor bem configurado transforma-te num programador mais rápido e menos propenso a erros. No VSCode, abre as Configurações (Ctrl+,) e procura por “format on save”. Ativa a opção para que o código seja formatado automaticamente sempre que guardares o ficheiro – isso mantém o teu código limpo e legível. Se estás a usar Python, instala a extensão “Black” e define-a como formatador padrão.

Outra funcionalidade que não deves ignorar é o terminal integrado. No VSCode, abre-o com Ctrl+` e escolhe o teu shell preferido (PowerShell, Bash ou CMD). Assim, podes compilar, testar e correr o teu código sem sair do editor. Configura-o para abrir na pasta do teu projeto por padrão; isso poupa cliques e mantém-te focado.

Para quem gosta de personalizar o visual, escolhe um tema que não canse os olhos. Temas escuros como “One Dark Pro” são populares porque reduzem o brilho e ajudam a concentração. Instala o tema, aplica-o e sente a diferença. Não é só estética; um fundo agradável evita fadiga visual durante sessões longas.

## Organiza os teus projetos desde o início

A forma como guardas os teus ficheiros determina a facilidade de encontrar e reutilizar código. Cria uma estrutura de pastas que reflita a lógica dos teus projetos. Por exemplo, numa aplicação web simples, poderias ter:

```
`` /meu-projeto | |— /src # código fonte | |— main.py | |—
utils.py |— /tests # testes unitários | |— test_main.py |—
requirements.txt # dependências |— README.md # descrição do
projeto ```
```

Este padrão de organização mantém tudo ordenado e permite que outras pessoas (ou tu mesmo no futuro) entendam rapidamente o que

está onde. Quando começares um novo projeto, cria um repositório Git vazio (``git init``) dentro da pasta raiz, faz o primeiro commit e estabelece uma branch “main”. Assim, já tens controlo de versão desde o primeiro dia.

Se preferires trabalhar com múltiplos projetos, cria uma pasta “Projetos” no teu diretório home e guarda-a lá. Evita colocar ficheiros de trabalho em pastas de documentos pessoais ou na área de trabalho; assim, quando fizeres backup, tudo ficará agrupado e será mais fácil de restaurar.

## **Mantém o teu ambiente limpo e atualizado**

Um ambiente de desenvolvimento que envelhece sem manutenção rapidamente torna-se um peso. Atualiza o teu sistema operativo regularmente – no Linux, corre ``sudo apt update && sudo apt upgrade``. No Windows, verifica as atualizações nas Configurações. Mantém o VSCode e as extensões ao dia; as extensões recebem correções de segurança e novas funcionalidades que podem melhorar a tua experiência.

Quando instalas novas bibliotecas, usa ambientes virtuais. No Python, cria-um com ``python -m venv .venv`` dentro da pasta do projeto e ativa-o (``source .venv/bin/activate`` no Linux ou ``.venv\Scripts\activate`` no Windows). Isso garante que cada projeto tem as suas dependências isoladas, evitando conflitos entre versões. Para JavaScript, o ``npm init`` cria um ``package.json`` que lista as dependências; usa ``npm install`` para gerir as versões.

Se alguma coisa deixa de funcionar, recorre ao terminal para diagnosticar. Mensagens de erro são os teus melhores amigos; lê-as com atenção, procura-as no Google (ou nos fóruns da comunidade) e

aprende a resolver problemas por ti mesmo. Essa prática desenvolve a autoconfiança e reduz a dependência de tutoriais.

### **FAZ ISTO HOJE**

Instala um sistema operativo (Linux ou Windows) no teu portátil se ainda não o fizeste.

Configura o Git com o teu nome e e-mail.

Descarrega e instala o VS Code e, dentro dele, a extensão da linguagem que vais usar.

Cria uma pasta chamada “Projetos” no teu diretório home e dentro dela um sub-directório chamado “primeiro-projeto”.

Dentro de “primeiro-projeto”, inicializa um repositório Git e cria a estrutura de pastas ``/src``, ``/tests``, ``README.md``.

Faz o primeiro commit com a mensagem “Inicialização do projeto”.

## CAPÍTULO 3

# Lógica de programação: o pensamento passo a passo

---

O sol já alta no bairro de Viana, e o som das rodas do candongueiro atravessa a rua enquanto os vendedores do Mercado da Póvoa já gritam os preços das frutas. Tu estás na fila da padaria, com a mão a bater no bolso à procura das tuas duas kwanzas que deixaste ali para pagar o pão. De repente, o senhor que está à tua frente pára e diz, “Se eu te der um pão e um pastel, quantas moedas ainda precisas para chegar a cinco?” Ele te olha, esperando a resposta. Nesse instante percebes que, sem sequer abrir um caderno, já estás a fazer contas, a dividir o problema em partes e a seguir um raciocínio lógico. Essa troca simples é a porta de entrada para a lógica de programação: um pensamento passo a passo que transforma um problema confuso num conjunto de instruções claras e executáveis.

## O que é lógica de programação

Lógica de programação não é um bicho de sete cabeças; é apenas a forma de organizar o pensamento para que uma máquina – ou mesmo o teu cérebro – possa seguir instruções sem se perder. Quando escreves um código, estás a escrever uma receita. Cada linha tem de ter sentido, de estar no lugar certo, de respeitar a ordem dos passos. Se a receita de um bolo pede “bater os ovos” antes de “quebrar os ovos”, o resultado será um desastre. Da mesma forma, se um programa tenta dividir por zero antes de verificar se o divisor é diferente de zero, o computador vai travar.

A lógica nasce da capacidade de observar um problema, perceber quais são as partes essenciais e transformar essas partes em ações sequenciais. Não há mistério: tudo o que precisas é de clareza e de dividir o todo em blocos menores que façam sentido. Quando aprendes a pensar desse jeito, tudo se torna mais fácil – desde calcular a troca de um mercado até criar um jogo que responde ao teu teclado.

## **Decompor um problema**

A primeira técnica que todo programador usa é a decomposição. Em vez de olhar para o problema como um monólito, quebra-o em sub-problemas. Imagina que tens de organizar uma lista de nomes de colegas da escola para uma festa. Em vez de tentar ordenar tudo de uma vez, pensa nos passos:

1. Ler a lista.
2. Verificar se há nomes repetidos.
3. Ordenar alfabeticamente.
4. Imprimir o resultado.

Cada um desses passos pode ser transformado em uma função ou bloco de código. O segredo está em manter a descrição de cada passo tão simples que até um amigo que nunca programou consiga entender.

Num cenário real, digamos que a tua avó te pediu para ajudar a contar o dinheiro que tem na caixa. Em vez de pegar todas as notas e moedas e tentar somar tudo de cabeça, podes dividir a tarefa:

- Separe as notas de 100, de 50 e de 20.
- Conte quantas há de cada.
- Multiplique o número de notas pelo valor correspondente.
- Some os resultados.

Ao seguir esse método, não só evitas erros, como também cria uma sequência lógica que pode ser escrita em código num piscar de olhos.

## Estruturas de decisão simples

Depois de dividir o problema, chega o momento de decidir o que fazer em cada situação. As estruturas de decisão – os famosos “if-else” – são a forma de dizer ao programa: “Se isto acontecer, faz aquilo; caso contrário, faz outra coisa”. Volta ao candongueiro: o senhor que te perguntou sobre as moedas tem duas respostas possíveis. Se tens exatamente duas kwanzas, a resposta é “zero”; se tens menos, a resposta muda.

Em código, isso fica algo como:

```
```python if moedas == 2: resposta = 0 else: resposta = 5 - moedas
```
```

A lógica aqui é direta: verifica a condição, escolhe o caminho e continua. O truque está em escolher a condição correta. Muitas vezes a confusão nasce porque a condição é escrita de forma ambígua ou porque o programa tem caminhos que nunca são testados. Por isso, antes de escrever o código, pergunta-te: “Qual é a pergunta que estou a fazer ao computador?” e “Qual será a resposta esperada para cada caso?” Quando essas perguntas ficam claras, as decisões surgem naturalmente.

## Pensar passo a passo – o algoritmo na prática

Um algoritmo é apenas uma lista de passos bem definidos. Não tem de ser complicado. Vamos pegar num exemplo do dia a dia: preparar uma garrafa de água para a aula. O que precisas fazer?

1. Encontra a garrafa vazia.
2. Leva-a ao bebedouro.
3. Abre o bebedouro.
4. Enche a garrafa até ficar cheia.
5. Fecha o bebedouro.
6. Volta à sala.

Cada passo tem um início e um fim, e todos são necessários para alcançar o objetivo. Se lhe faltasse um passo – por exemplo, fechar o bebedouro – o resultado seria uma garrafa que continua a pingar, desperdiçando água.

Quando traduzes esse algoritmo para código, a ideia permanece a mesma. Por exemplo, num programa que calcula a média de três notas, o algoritmo seria:

1. Ler a primeira nota.
2. Ler a segunda nota.
3. Ler a terceira nota.
4. Somar as três notas.
5. Dividir a soma por três.
6. Mostrar o resultado.

Em Python, ficaria algo como:

```
```python nota1 = float(input("Primeira nota: ")) nota2 = float(input("Segunda nota: ")) nota3 = float(input("Terceira nota: "))  
  
soma = nota1 + nota2 + nota3 media = soma / 3  
  
print("A média é:", media) ```
```

Repara como cada linha corresponde exatamente a um passo do algoritmo. Essa correspondência ajuda a depurar: se o resultado não for o esperado, volta ao passo que pode estar errado, em vez de procurar o erro em todo o código.

O ciclo de “pensar-escrever-testar”

A prática de programar não termina quando guardas o ficheiro. O ciclo de “pensar-escrever-testar” faz parte da lógica de programação. Primeiro, pensas no algoritmo; depois, escreves o código que o

implementa; por fim, testas com dados reais. Se algo falha, volta ao pensamento e ajusta o algoritmo. Esse ciclo cria uma mentalidade de melhoria contínua e evita que te percas em detalhes antes de entender o todo.

O papel da abstração

Abstrair significa focar nos detalhes relevantes e esconder o resto. Quando trabalhas com listas de nomes, não precisas de saber como o computador armazena cada nome; basta saber que podes percorrer a lista, comparar valores e acrescentar novos elementos. A abstração permite que te concentres no problema de alto nível, enquanto a máquina cuida dos pormenores. É como ao cozinhar: não precisas de conhecer a química de cada ingrediente para seguir a receita; basta saber o que cada passo faz.

Aplicar a lógica no teu próximo projeto

Agora que já sabes como dividir um problema, decidir caminhos e transformar tudo em passos claros, chega a hora de aplicar isso num projeto que faça sentido para ti. Pensa numa situação que enfrentas todos os dias – talvez organizar a lista de compras para a merenda da escola, ou calcular quanto tempo de viagem precisas para chegar ao trabalho se o trânsito está lento. Escolhe uma dessas situações e escreve um algoritmo simples. Depois, abre o teu editor de código, cria um ficheiro, e transforma o algoritmo em código. Não te preocupes se o programa ainda não funciona perfeitamente; o objetivo é praticar o pensamento passo a passo.

A lógica de programação é a ponte entre a tua mente e a máquina. Quanto mais a utilizares, mais natural se tornará o processo de transformar uma ideia em algo que funciona. E, como qualquer habilidade, a prática constante é a chave para o domínio.

FAZ ISTO HOJE

Escolhe um problema cotidiano que te incomoda (por exemplo, calcular o troco de uma compra ou organizar uma lista de tarefas).

Escreve, num papel, os passos que precisas seguir para resolver esse problema, um por linha.

Transforma cada passo num comando de Python (ou outra linguagem que já conheças).

Executa o programa e verifica se o resultado corresponde ao esperado.

Se algo não funcionar, volta ao passo que parece errado, ajusta a lógica e testa novamente.

CAPÍTULO 4

Variáveis, tipos e operadores

Era uma manhã de segunda-feira no bairro da Luanda, Talatona. O teu irmão mais novo, Carlos, estava sentado no chão da cozinha, com o telemóvel na mão, a tentar descobrir por que razão o seu jogo de perguntas-e-respostas não aceitava a palavra “cachimbo”. Enquanto a mamãe preparava o pequeno-almoço, eu ouvi-o dizer: “Se eu mudar ‘cachimbo’ por ‘cachimbo’, ainda não funciona!”. Lá fora, o candongueiro já fazia a ronda, e o som das rodas a bater no asfalto lembrava-te que o dia já tinha começado. Carlos estava a lidar com um conceito que ainda não tinha nome: **variável**. Ele tinha um valor que queria mudar, mas não sabia como falar a linguagem da máquina. Foi aí que percebi que, se eu lhe mostrasse como funcionam as variáveis, os tipos e os operadores, ele deixaria de lutar contra o código e passaria a brincar com ele.

O que são variáveis

Imagina que tens um caderno onde anotas o dinheiro que ganhas por cada venda de pastel de nata. Na primeira página escreves “dinheiro = 0”. Cada vez que vendes um pastel, somas 5 kwanzas ao total. A palavra “dinheiro” funciona como um **rótulo**: sempre que a lês, sabes a que número te referes. Na programação, esse rótulo chama-se **variável**. É um espaço na memória do computador que guarda um valor e que pode mudar ao longo da execução do programa.

Quando escreves ``dinheiro = dinheiro + 5``, estás a dizer ao computador: “encontra o valor que está guardado em ‘dinheiro’,

soma-le 5 e guarda o resultado de volta em ‘dinheiro’”. A palavra à esquerda do sinal de igualdade (o ``=``) é o **nome da variável**, e o que está à direita é a **expressão** que determina o novo valor. O nome tem de ser significativo – não escrevas ``x = 10`` se o que pretendes é representar a quantidade de kwanzas que tens. Um nome claro como ``saldoKwanzas`` já diz muito mais à primeira vista.

As variáveis não são “coisas” que surgem do nada; elas precisam ser **declaradas** antes de serem usadas. Em Python, por exemplo, basta atribuir um valor e a linguagem cria a variável automaticamente. Em linguagens como Java ou C, precisas de indicar o **tipo** da variável (mais detalhes adiante) antes de a usar: ``int saldoKwanzas;``. Se esqueceres a declaração, o programa falha antes mesmo de iniciar, como se o teu irmão tivessem de comprar um bilhete antes de subir ao candongueiro.

Tipos de dados

Um número pode ser **inteiro** (``int``) ou **decimal** (``float``). Se pretendes contar quantas frutas há numa sacola, usas um inteiro: ``frutas = 12``. Se precisas de medir a distância percorrida em quilómetros com duas casas decimais, usas um decimal: ``distancia = 3.75``. O tipo determina as operações que podes fazer com aquele valor. Não faz sentido dividir um número inteiro por outro e esperar um resultado decimal se a linguagem só aceita inteiros; o resto da divisão seria perdido.

Além de números, há **cadeias de texto** (``str``). São sequências de caracteres que representam palavras, frases, ou até mesmo números quando não precisas de fazer cálculos. Se quiseres guardar o nome do teu colega de equipa, escreves ``nomeColega = "Ana"`` – as aspas delimitam a cadeia. Cuidado: ``"123"`` não é o mesmo que ``123``. O

primeiro é texto, o segundo é número. Essa distinção pode salvar-te de bugs difíceis de rastrear.

Outro tipo útil são os **booleanos** (``bool``). Eles só podem valer ``True`` ou ``False``. Servem para decisões: “o utilizador já está autenticado?” (``autenticado = True``). Quando combinados com operadores lógicos (veremos mais abaixo), permitem construir condições complexas de forma legível.

Python ainda tem colecções como **listas** (``list``) e **dicionários** (``dict``). Uma lista guarda uma sequência de valores, por exemplo ``notas = [15, 18, 12]``. Já um dicionário associa pares chave-valor: ``precoProdutos = {"pastel": 5, "café": 3}``. Não precisas de entrar em pormenores agora, mas saberes que as variáveis podem conter outros tipos de dados abre muitas portas para resolver problemas do dia a dia.

Operadores: aritmética e lógica

Os **operadores** são os símbolos que dizem ao computador como combinar valores. Os mais familiares são os aritméticos: ``+``, ``-``, ``*``, ``/`` e ``%`` (resto da divisão). Se tens ``saldoKwanzas = 20`` e vendes um pastel a 5 kwanzas, a operação ``saldoKwanzas = saldoKwanzas - 5`` devolve 15. O operador ``%`` é útil quando precisas de saber se um número é par ou ímpar: ``if numero % 2 == 0:`` indica que o número tem resto 0 quando dividido por 2, logo é par.

Existem ainda operadores **de comparação** que retornam booleanos: ``==`` (igual), ``!=`` (diferente), ``<``, ``>``, ``<=``, ``>=``. Eles são a base das decisões no teu código. Por exemplo, para garantir que o utilizador não introduz um número negativo de kwanzas, poderias escrever:

```
```python if quantidadeKwanzas < 0: print("Não pode inserir valor negativo") ```
```

A linguagem avalia a condição e, se for ``True``, executa o bloco indentado.

Os **operadores lógicos** (``and``, ``or``, ``not``) permitem combinar várias condições. Se quiseres validar que o utilizador introduziu tanto um nome quanto um saldo positivo, usarás:

```
```python if nome != "" and saldoKwanzas > 0: print("Tudo pronto") ```
```

O ``and`` exige que ambas as condições sejam verdadeiras; o ``or`` aceita que bastasse uma delas; o ``not`` inverte o resultado (``not False`` vira ``True``). Estes operadores dão ao teu código a capacidade de raciocinar como um ser humano: “se a pessoa tem nome e tem dinheiro, então pode comprar”.

Boas práticas ao usar variáveis

- **Nomes claros:** como já mencionei, um nome como ``quantidadeKwanzas`` comunica instantaneamente a sua finalidade. Evita abreviações obscuras; se não ficaste a lembrar o que significa ``qtKz``, o teu futuro eu vai precisar de tempo para descobrir.
- **Consistência:** escolhe um estilo (`snake_case`, que usa underscores, ou `camelCase`) e mantém-o em todo o projecto. Em Python, o padrão é `snake_case`, como ``saldo_kwanzas``.
- **Escopo:** lembra-te de onde a variável foi criada. Se a declares dentro de uma função, só será visível ali. Variáveis globais podem ser úteis, mas abusar delas torna o código difícil de seguir, tal como um candongueiro que tenta levar toda a carga de todo o bairro ao mesmo tempo.

- **Inicialização:** sempre dá um valor inicial à variável. Se deixares ``saldoKwanzas`` sem definir, o programa pode falhar ou usar um valor inesperado. Um ``saldoKwanzas = 0`` deixa claro que o ponto de partida é zero.
- **Tipo adequado:** não guardes números como texto a não ser que seja realmente necessário. Se ``saldoKwanzas`` for uma string, não poderás somar nem subtrair sem converter primeiro, o que gera código extra e risco de erros.

Ao dominar estas regras, transformarás o teu código num mapa claro, onde cada variável tem a sua placa de sinalização. Quando o teu irmão Carlos vir que pode mudar o valor de ``palavra`` num jogo simplesmente atribuindo ``palavra = "cachimbo"`` e que o programa reage de acordo, ele entenderá que o poder está nas variáveis, nos tipos e nos operadores que as manipulam.

FAZ ISTO HOJE

Cria um ficheiro chamado ``exercicio_variaveis.py``.

Dentro do ficheiro, declara três variáveis: ``saldoKwanzas``, ``nomeProduto`` e ``emStock``.

Atribui a ``saldoKwanzas`` o valor 50, a ``nomeProduto`` a string ``"pastel"`` e a ``emStock`` o valor booleano ``True``.

Escreve uma linha que aumente ``saldoKwanzas`` em 5 quando ``emStock`` for ``True``.

Imprime no ecrã a frase: “Tem X kwanzas disponíveis para vender Y”, substituindo X pelo saldo atualizado e Y pelo nome do produto.

Por fim, adiciona uma condição que, se ``saldoKwanzas`` for maior que 100, mostre a mensagem “Saldo elevado, atenção ao limite”.

CAPÍTULO 5

Estruturas de controlo: loops e decisões

O dia estava quente em Cacuaco. O sol a pino fazia o asfalto do bairro do Kilamba brilhar como se fosse prata. Eu estava sentado num banco de cimento junto ao mercado da rua da Lúcia, enquanto o candongueiro do João, já cansado, carregava duas caixas de farinha de milho para vender. De repente, a filha do senhor João, a Maria, aproximou-se com um telemóvel na mão e disse: “Pai, preciso de ajuda para fazer um programa que conte quantas vezes os clientes comprem pão”. O João riu, apertou a mão da filha e respondeu: “Então vamos aprender a programar, que a vida não espera”. Ali, naquele momento, percebi que a lógica dos algoritmos podia estar tão presente nas nossas rotinas como a fila no mercado ou o barulho dos motores dos candongueiros.

Decisões: quando escolher um caminho

Num programa, a primeira coisa que te permite mudar o rumo da execução são as estruturas de decisão. Em termos simples, elas são como os sinais de trânsito da tua vida: se o semáforo está verde, avanças; se está vermelho, esperas. Em Python, a palavra-chave `if` abre a porta para essa escolha. Por exemplo, imagina que o João quer saber se o número de pães vendidos naquele dia ultrapassa a meta de 50. O código ficaria assim:

```
python pães_vendidos = 57 meta = 50
```

```
if pães_vendidos > meta: print("Meta alcançada!") else:  
print("Ainda falta vender.") ``
```

A palavra ``else`` cobre o caso contrário, mas há situações em que precisas de mais de duas opções. É aqui que entra o ``elif`` (abreviação de “else if”). Supõe que o João queira saber se vendeu menos de 30, entre 30 e 50 ou mais de 50 pães. O programa ficaria:

```
``python if pães_vendidos < 30: print("Venda baixa") elif  
pães_vendidos <= 50: print("Venda média") else: print("Venda alta")  
``
```

Repara que as condições são avaliadas de cima para baixo; assim que uma delas for verdadeira, o resto é ignorado. Essa característica pode ser usada a teu favor para otimizar a execução, mas também pode surpreender se não fores cuidadoso. Por isso, pensa sempre em que ordem faz sentido para o teu problema.

Um detalhe que costuma passar despercebido é que a expressão que segue ao ``if`` tem de ser booleana – isto é, tem de devolver ``True`` ou ``False``. Qualquer coisa que não seja explicitamente booleana será convertida para um valor booleano, mas confiar nessa conversão pode gerar bugs difíceis de perceber. Por isso, quando estiveres a comparar números, strings ou listas, usa operadores como ``==``, ``!=``, ``<``, ``>``, ``<=`` ou ``>=``. Evita misturar tipos diferentes; comparar um número com uma string pode dar resultados inesperados.

Loops: repetir até que algo aconteça

A vida tem ciclos – acordar, ir ao trabalho, voltar para casa – e o mesmo acontece nos programas. Quando precisas de executar a mesma instrução várias vezes, recorres aos loops. Em Python, os dois tipos mais usados são o ``while`` e o ``for``.

O loop `while`

O `while` repete um bloco enquanto a condição for verdadeira. Imagina que o João tem um saco de grãos e quer contar quantas vezes consegue retirar 5 kg até esgotar o saco. O código seria:

```
```python
saco = 120 # quilogramas
contagem = 0

while saco >= 5:
 saco -= 5
 contagem += 1
 print("Retiradas feitas:", contagem)
```
```

Note que a condição `saco >= 5` é avaliada a cada iteração. Se esqueceres de atualizar a variável que controla a condição, o loop pode ficar infinito, bloqueando o teu programa. Para evitar isso, verifica sempre se há um caminho para que a condição se torne falsa.

O loop `for`

O `for` é mais adequado quando sabes de antemão quantas vezes o ciclo deve acontecer, como ao percorrer uma lista de clientes. Suponhamos que tens a lista dos nomes dos clientes que compraram pão hoje:

```
```python
clientes = ["Ana", "Beto", "Carlos", "Diana"]
for nome in clientes:
 print(nome, "comprou pão")
```
```

Aqui, a variável `nome` recebe, a cada iteração, o próximo elemento da lista. O `for` também aceita a função `range`, que gera uma sequência de números. Se quiseres imprimir os primeiros dez números naturais, faz:

```
```python
for i in range(1, 11):
 print(i)
```
```

Um truque útil é combinar `for` com `enumerate` quando precisas do índice do elemento além do próprio valor. Isso pode ser útil para gerar um relatório onde cada linha tem um número de ordem:

```
```python for indice, nome in enumerate(clientes, start=1): print(f'
{indice}. {nome}') ```
```

### Quebrar e continuar

Durante um loop, pode surgir a necessidade de interromper a iteração antes de terminar ou saltar para a próxima. As palavras-chave `break` e `continue` servem exatamente a esse propósito. Imagina que o João tem um cliente que não compra pão porque tem alergia ao glúten; ao encontrar esse cliente na lista, quer interromper a contagem:

```
```python clientes = ["Ana", "Beto", "Carlos", "Diana", "Ema"] for
nome in clientes: if nome == "Ema": break print(nome, "comprou
pão") ```
```

Neste caso, o programa deixa de imprimir “Ema comprou pão”. O `continue` funciona de forma oposta – pula o resto do bloco e passa à próxima iteração. Se o João quiser ignorar o cliente “Carlos” mas continuar a contar os restantes, faria:

```
```python for nome in clientes: if nome == "Carlos": continue
print(nome, "comprou pão") ```
```

## Aninhamento: decisões dentro de loops

Muitas vezes, decisões e loops não permanecem isolados; eles se combinam para resolver problemas mais complexos. Por exemplo, o João pode querer saber quantos clientes compraram mais de 2 unidades de pão. Primeiro, tem que percorrer a lista de clientes, depois, para cada cliente, verificar a quantidade comprada. Um modo de fazer isso é aninhar um `if` dentro de um `for`:

```
```python compras = { "Ana": 1, "Beto": 3, "Carlos": 2, "Diana": 4,
"Ema": 0 }
```

```
conta_maiores = 0
for cliente, quantidade in compras.items():
    if quantidade > 2:
        conta_maiores += 1
print("Clientes que compraram mais de 2 pães:", conta_maiores) ````
```

Neste exemplo, o dicionário `compras` associa cada cliente a uma quantidade. O método `items()` devolve pares `(chave, valor)`, permitindo que o `for` atribua simultaneamente o nome do cliente e a quantidade comprada. O `if` interno filtra apenas os casos que satisfazem a condição `quantidade > 2`. O resultado final é a soma desses casos.

Um ponto de atenção aqui é a legibilidade. Quando aninhas múltiplos `if` e `for`, o código pode ficar difícil de seguir. Uma boa prática é usar indentação clara e, quando possível, dividir a lógica em funções menores. Isso não só melhora a compreensão, como também facilita a manutenção – algo que vais precisar quando o teu código crescer e começar a ser usado por outras pessoas.

Boas práticas para loops e decisões

Não há necessidade de transformar tudo em um único bloco gigante. Quando fores a usar loops, pensa primeiro se precisas mesmo de repetir algo ou se há uma forma mais direta de alcançar o objetivo. Por vezes, uma simples compreensão de lista pode substituir um `for` completo, tornando o código mais conciso. Por exemplo, a lista de clientes que compraram mais de duas unidades pode ser obtida assim:

```
python clientes_maiores = [c for c, q in compras.items() if q > 2]
print(len(clientes_maiores)) ````
```

Além disso, evita loops infinitos a todo custo. Sempre verifica se a condição que controla o `while` será alterada dentro do bloco. Se houver risco de um loop ficar preso, considera usar um contador de segurança que força a saída após um número máximo de iterações.

Por fim, não te esqueças de comentar o teu código quando a lógica não for imediatamente óbvia. Um comentário curto pode poupar horas de depuração mais tarde.

FAZ ISTO HOJE

Cria um pequeno programa que simule a venda de pães num mercado. Define uma lista com os nomes de 10 clientes e, para cada cliente, gera aleatoriamente um número entre 0 e 5 que represente a quantidade de pães comprados. Usa um loop ``for`` para percorrer a lista, um ``if`` para contar quantos clientes compraram mais de 2 pães e, no final, imprime esse total.

Não te esqueças de usar ``break`` para interromper o loop caso um cliente compre exatamente 5 pães, e ``continue`` para saltar a contagem quando a quantidade for 0.

Testa o programa várias vezes para perceber como a aleatoriedade altera o resultado.

CAPÍTULO 6

Funções: reutiliza o teu código

Amanhã, no Mercado da Baía, o teu primo João vendeu-te um candongueiro de 5kwanzas e, entre uma compra e outra, ele começou a falar sobre a última aplicação que tinha criado para ajudar a contar as vendas diárias. “Olha, se eu tivesse uma maneira de não ter de escrever o mesmo cálculo toda a hora, teria ganho mais tempo para conversar com a gente”, disse ele, esfregando as mãos no peito enquanto mostrava um pequeno programa no telemóvel. Naquele instante percebeste que o que ele precisava era de um “atalho”. Esse atalho, no mundo da programação, chama-se **função**.

O que são funções

Uma função é basicamente um bloco de código que pode ser chamado tantas vezes quanto precisares. Imagina que tens de calcular o imposto de vendas para cada artigo que vendes. Em vez de copiar-colar a mesma sequência de linhas para cada produto, escreves uma única sequência, dás-lhe um nome e depois “invocas” esse nome sempre que quiseres fazer o cálculo. Assim, não só reduces a quantidade de código, como também facilitas a manutenção: se houver mudança na taxa de imposto, alteras o cálculo num único ponto e o resto do programa já usa a nova fórmula.

Definir a tua primeira função

Em Python, que é a linguagem que vamos usar neste livro, a definição de uma função começa com a palavra-chave ``def``, seguida do nome

que escolheste, dos parênteses e dois-pontos. Dentro dos parênteses, podes colocar os **parâmetros** – os valores que a função vai receber quando for chamada. Depois, o corpo da função, indentado, contém as instruções que serão executadas.

```
```python def calcula_imposto(preco): taxa = 0.14 # 14% de imposto
retorno = preco * taxa return retorno ```
```

A palavra ``return`` indica o que a função devolve ao código que a chamou. Sem ``return``, a função executa o seu bloco, mas não entrega nenhum valor utilizável. No exemplo acima, ``calcula_imposto`` recebe o preço de um artigo e devolve o valor do imposto.

Para usar a função, basta chamar o seu nome e passar o argumento necessário:

```
```python preco_camisa = 1200 imposto_camisa = calcula_imposto(preco_camisa)
print(imposto_camisa) # 168.0 ```
```

Note que a variável ``preco_camisa`` não precisa de ser chamada dentro da função; o que importa é que o valor 1200 seja passado para o parâmetro ``preco``.

Parâmetros e argumentos

Os parâmetros são como variáveis que existem apenas dentro da função. Quando chamas a função, os argumentos que forneces são “copiados” para esses parâmetros. Podes ter quantos parâmetros quiseres, separados por vírgulas. Se a tua aplicação precisar de mais de um cálculo – por exemplo, o imposto e o lucro – podes criar uma função que devolva ambos.

```
```python def calcula_imposto_e_lucro(preco, margem_lucro): taxa = 0.14
imposto = preco * taxa lucro = preco * margem_lucro return imposto, lucro ```
```

Ao chamar a função, recibes uma tupla com dois valores:

```
```python imposto, lucro = calcula_imposto_e_lucro(1500, 0.30)
print(imposto) # 210.0 print(lucro) # 450.0 ```
```

A flexibilidade dos parâmetros permite que a mesma função sirva para diferentes situações. Se o teu primo João quiser mudar a taxa de imposto para 16%, basta alterar a linha `taxa = 0.14`` dentro da função – todo o resto do código que a usa continuará a funcionar.

Funções que retornam valores

Um dos grandes poderes das funções está na capacidade de retornar valores que podem ser usados imediatamente ou armazenados para uso posterior. Por vezes, precisas de fazer várias operações dentro de uma função e só então devolver o resultado final.

```
```python def total_com_imposto(preco, taxa=0.14): imposto =
preco * taxa total = preco + imposto return total ```
```

Aqui, a taxa tem um valor padrão (0.14). Se não passes nenhum segundo argumento, a função usa esse valor; se o passares, substitui-o.

```
```python print(total_com_imposto(2000)) # 2280.0 (taxa
padrão) print(total_com_imposto(2000, 0.16)) # 2320.0 (taxa
alterada) ```
```

Esse mecanismo de valores padrão reduz a necessidade de escrever código extra para verificar se um argumento foi fornecido ou não.

Reutilizando código com funções

A verdadeira magia acontece quando comes a usar funções em projetos maiores. Imagina que o teu primo decide criar um pequeno sistema para registrar todas as vendas do dia, incluindo o preço, o

imposto e o lucro. Em vez de escrever as mesmas linhas de cálculo para cada linha da tabela, cria funções para cada passo e chama-as sequencialmente.

```
```python def registra_venda(nome_artigo, preco): imposto =
calcula_imposto(preco) lucro = calcula_imposto_e_lucro(preco,
0.30)[1] total = preco + imposto print(f'Artigo: {nome_artigo}')
print(f'Preço: {preco} Kz') print(f'Imposto: {imposto:.2f} Kz')
print(f'Lucro: {lucro:.2f} Kz') print(f'Total a cobrar: {total:.2f} Kz')
print('---') ```
```

Agora, basta chamar `registra\_venda` sempre que houver uma nova transação:

```
```python          registra_venda('Camiseta',          1200)
registra_venda('Calça', 2500) ```
```

Cada chamada reutiliza as funções `calcula\_imposto` e `calcula\_imposto\_e\_lucro`, mantendo o código limpo e fácil de entender. Se, algum dia, o teu primo quiser acrescentar um novo cálculo, como um desconto, basta criar outra função e incluí-la dentro de `registra\_venda`.

Funções aninhadas e escopo

Um ponto que costuma confundir iniciantes é o **escopo** – onde as variáveis existem e podem ser acedidas. As variáveis definidas dentro de uma função são locais a essa função; não podem ser vistas fora dela. Se tentares usar `imposto` fora de `registra\_venda`, receberás um erro.

```
```python def exemplo(): local = 10 return local

print(exemplo()) # 10 # print(local) # erro: name 'local' is not
defined ```
```

Essa separação protege o teu código de interferências inesperadas e permite que duas funções usem variáveis com o mesmo nome sem conflito.

## Funções como blocos de construção

Ao longo da tua jornada de programador, notarás que as funções são os blocos de construção dos programas mais complexos. Elas permitem dividir um problema grande em partes menores, cada uma responsável por uma tarefa específica. Essa abordagem, conhecida como **programação modular**, facilita a colaboração: diferentes programadores podem trabalhar em funções distintas e depois juntar-as num único projeto.

Pensa no teu futuro portefólio. Se consegues apresentar projetos onde o código está bem organizado em funções, demonstras não só domínio da linguagem, mas também capacidade de pensar de forma estruturada – algo que empregadores valorizam muito.

## Boas práticas ao criar funções

- Dá nomes claros e descritivos. Em vez de ``f1``, usa ``calcula_imposto``.
- Mantém as funções curtas. Se uma função começa a crescer demais, talvez esteja a fazer mais do que deveria.
- Evita efeitos colaterais inesperados, como mudar variáveis globais sem necessidade.

Seguindo estas recomendações, as tuas funções serão fáceis de ler, de testar e de reutilizar.

### FAZ ISTO HOJE

Cria um ficheiro chamado ``vendas.py``.

Define a função ``calcula_imposto`` que aceita um preço e devolve o imposto a 14 %.

Define a função ``calcula_lucro`` que aceita um preço e uma margem (por exemplo, 0.30) e devolve o lucro.

Define a função ``registra_venda`` que recebe o nome do artigo e o preço, usa as duas funções anteriores para calcular imposto e lucro, e imprime um resumo formatado como no exemplo acima.

Chama ``registra_venda`` três vezes com artigos diferentes e preços à tua escolha.

## CAPÍTULO 7

# Trabalhar com dados: listas e dicionários

---

Na esquina da Rua da Sé, junto à banca de frutas do João, o teu primo Carlos vendia garrafas de água de 20 cl. Cada manhã, antes de abrir a loja da avó no Bairro da Várzea, ele sentava-se ao portão, ligava o telemóvel e, entre um gole e outro, verificava as vendas da semana passada. Na última segunda-feira, ao abrir a planilha no Excel, percebeu que o número de garrafas vendidas variava muito de dia para dia. “Se eu soubesse organizar esses números...” murmurou, enquanto os clientes passavam apressados. Foi aí que te perguntou se sabias alguma coisa sobre “listas” e “dicionários”. O teu primeiro contacto com a programação não foi num laboratório, mas na mesma rua onde vendes pão na padaria da tua mãe. Esta história mostra que, mesmo nos cantos mais simples de Luanda, os dados estão à espera de quem os entende.

### Entender listas

Uma lista é, basicamente, um conjunto de elementos guardados na ordem em que os inserimos. Imagina-te a fila dos clientes que entram no mercado do Candongueiro: o primeiro a entrar, o segundo a entrar... Se criasses uma lista com esses nomes, o Python manteria exatamente essa sequência.

```
```python clientes = ["Ana", "Bento", "Carla", "Dário"]```
```

A palavra-chave aqui não é “coletar”. É “manter”. Cada elemento tem um índice, começando em zero. Quando queres saber quem está na terceira posição, não precisas de contar manualmente; basta:

```
```python terceiro = clientes[2] # "Carla" ```
```

Se quiseres acrescentar alguém ao final da fila, usas o método `append`:

```
```python clientes.append("Ema") ```
```

E se, de repente, alguém desistiu de comprar, basta retirar o nome:

```
```python clientes.remove("Bento") ```
```

A beleza das listas está na flexibilidade. Podes guardar números, textos, ou até outras listas. Por exemplo, para acompanhar o stock de garrafas de água que o Carlos vende, poderias fazer:

```
```python vendas_diarias = [15, 22, 19, 30, 12] # quantidade  
vendida em cada dia da semana ```
```

A partir daí, tudo o que precisas fazer é usar operações simples para obter a informação que te interessa.

Operações comuns com listas

Contar e somar

Para saber quantas garrafas foram vendidas ao todo, basta somar os valores da lista:

```
```python total_vendas = sum(vendas_diarias) # 98 ```
```

Se quiseres descobrir quantas vezes uma quantidade específica apareceu, usas `count`:

```
```python vezes_22 = vendas_diarias.count(22) # 1 ```
```

Ordenar e filtrar

Talvez queiras ver os dias em que as vendas foram superiores a 20. Em vez de percorrer a lista à mão, podes criar uma nova lista com apenas esses valores:

```
```python acima_de_20 = [v for v in vendas_diarias if v > 20] #  
[22, 30] ```
```

Se quiseses apresentar os dados do maior para o menor, a função `sorted` faz o trabalho:

```
```python vendas_ordenadas = sorted(vendas_diarias,  
reverse=True) # [30, 22, 19, 15, 12] ```
```

Acessar fatias

Às vezes, só precisas dos primeiros três dias da semana. Python permite “fatiar” listas:

```
```python primeiros_tres = vendas_diarias[:3] # [15, 22, 19] ```
```

E, se quiseses tudo exceto o primeiro dia, usas:

```
```python sem_primeiro = vendas_diarias[1:] # [22, 19, 30, 12]  
```
```

Essas pequenas ferramentas já dão-te poder para transformar um monte de números em informação útil. O próximo passo é aprender a atribuir significado a cada número, e é aqui que entram os dicionários.

## Dicionários: o teu mapa de dados

Um dicionário funciona como um mapa de rua. Em vez de usar apenas um índice numérico, associamos cada chave a um valor. No

caso do Carlos, cada dia da semana pode ser a chave, e o número de garrafas vendidas, o valor.

```
```python vendas_por_dia = { "segunda": 15, "terça": 22, "quarta":  
19, "quinta": 30, "sexta": 12 } ```
```

A grande vantagem é a legibilidade. Quando lês `vendas_por_dia["quinta"]`, já sabes que estás a aceder ao número de garrafas vendidas na quinta-feira, sem precisar de lembrar que o índice 3 corresponde a esse dia.

Acrescentar e atualizar

Se amanhã vendeste 18 garrafas, basta fazer:

```
```python vendas_por_dia["sábado"] = 18 ```
```

Se precisas de corrigir um erro – digamos que na quarta realmente vendeste 20 em vez de 19 – atualizas assim:

```
```python vendas_por_dia["quarta"] = 20 ```
```

Percorrer um dicionário

Para imprimir todas as vendas da semana, podes iterar sobre o dicionário:

```
```python for dia, quantidade in vendas_por_dia.items(): print(f'  
{dia.title()}: {quantidade} garrafas') ```
```

O método `items()` devolve pares (chave, valor) que são muito úteis quando precisas de fazer cálculos ou gerar relatórios.

### Verificar a existência de uma chave

Antes de aceder a um dia que ainda não tem registo, é prudente confirmar se a chave está presente:

```
``python if "domingo" in vendas_por_dia:
print(vendas_por_dia["domingo"]) else: print("Ainda não há vendas
registradas para domingo.") ``
```

### Combinar dicionários

Se tens outro vendedor, a Maria, que também tem um dicionário com as suas vendas, podes combiná-los:

```
``python vendas_maria = {"segunda": 10, "terça": 14, "quarta": 11,
"quinta": 9, "sexta": 7} vendas_total = {dia: vendas_por_dia.get(dia,
o) + vendas_maria.get(dia, o) for dia in set(vendas_por_dia) |
set(vendas_maria)} ``
```

Agora `vendas\_total` contém a soma das vendas de ambos os vendedores para cada dia da semana. Essa técnica de “mesclar” dicionários será útil quando trabalhares com bases de dados maiores ou com várias fontes de informação.

## Combinar listas e dicionários

Muitas vezes, os dois tipos de estrutura andam de mãos dadas. Imagina que o Carlos quer guardar, além da quantidade vendida, o preço por garrafa de cada dia, porque às vezes ele faz promoções. Uma forma limpa de organizar isso é usar uma lista de dicionários, onde cada dicionário representa um dia.

```
``python historico = [{"dia": "segunda", "quantidade": 15,
"preco": 0.5}, {"dia": "terça", "quantidade": 22, "preco": 0.45}, {"dia":
"quarta", "quantidade": 20, "preco": 0.5}, {"dia": "quinta",
"quantidade": 30, "preco": 0.4}, {"dia": "sexta", "quantidade": 12,
"preco": 0.5}] ``
```

Com essa estrutura, podes calcular rapidamente o faturamento diário:

```
```python for registo in historico: faturamento =  
registo["quantidade"] * registo["preco"] print(f'{registo['dia'].title():  
{faturamento:.2f} Kz") ```
```

Se quiseres saber qual foi o dia com maior faturamento, usas a função `max` com uma chave personalizada:

```
```python dia_maior_faturamento = max(historico, key=lambda  
x: x["quantidade"] * x["preco"]) print(f'Maior faturamento foi na
{dia_maior_faturamento['dia'].title()}') ```
```

Essa combinação de listas e dicionários permite que cries relatórios mais sofisticados, sem perder a clareza. Cada registo mantém tudo o que precisas – dia, quantidade, preço – e a lista mantém a ordem cronológica, essencial para análises temporais.

### FAZ ISTO HOJE

Cria um ficheiro chamado `vendas.py`.

Dentro dele, define a lista `vendas\_diarias` com cinco números que representem a quantidade de garrafas vendidas numa semana.

Cria um dicionário `precos` onde cada dia da semana (segunda a sexta) tem um preço diferente (pode ser 0.5, 0.45, etc.).

Constrói uma lista de dicionários chamada `historico`, onde cada elemento tem as chaves `"dia"`, `"quantidade"` e `"preco"` usando os valores da lista e do dicionário.

Escreve um ciclo que calcule e imprima o faturamento diário e, no final, indique qual o dia com maior faturamento.

## CAPÍTULO 8

# Introdução ao desenvolvimento web

---

Era uma manhã de junho no bairro da Samba, na zona de Kinshasa, e o som das conversas no mercado de rua misturava-se ao claxon dos candongueiros que subiam as ladeiras. Tu encontraste-te ao lado da tia Maria, que vendia chapéus de palha, e ela, com a mão cheia de moedas de 200 kwanzas, perguntou-te se já tinhas pensado em criar um site para o seu negócio. Ela explicou-te que, quando alguém procura “chapéus de palha em Luanda” no telemóvel, o seu pequeno stand raramente aparece nos resultados. Tu sentaste-te naquele banco de madeira, o sol a aquecer a tua cara, e percebeste que a resposta podia estar nas linhas de código que ainda não tinhas escrito. Foi ali que decidiste mergulhar no mundo da web – não como observador, mas como criador.

## O que é a Web?

A Web não é só um conjunto de páginas bonitas; é um ecossistema onde computadores, servidores e navegadores falam entre si. Quando abres o Chrome no teu telemóvel, o navegador envia um pedido a um servidor que guarda os arquivos do site. Esse servidor devolve-te HTML, CSS e JavaScript, que o navegador interpreta e transforma num layout que reconheces – o texto, as imagens, os botões. Cada vez que carregas numa ligação, esse ciclo repete-se. Entenderes que a Web funciona como uma conversa te dá uma base sólida para começar a escrever as tuas próprias respostas.

## HTML: a estrutura da página

O HTML (HyperText Markup Language) é a fundação de qualquer página. Imagina-o como o esqueleto de um edifício; define onde estão as paredes, as janelas e as portas. Cada elemento tem a sua tag – ``<header>``, ``<nav>``, ``<section>``, ``<footer>`` – e essas tags dão sentido ao conteúdo. Quando criares a página da tia Maria, vais querer um título que diga “Chapéus de Palha – Feitos à Mão”, uma imagem do seu melhor chapéu, e um formulário simples para que os clientes deixem o contacto. O HTML permite-te colocar tudo isso em ordem, sem preocupação com a aparência ainda.

```
```html <!DOCTYPE html> <html lang="pt"> <head> <meta
charset="UTF-8"> <title>Chapéus de Palha – Tia Maria</title>
</head> <body> <header> <h1>Chapéus de Palha – Feitos à
Mão</h1> </header> <section>  <p>Descobre a tradição dos nossos chapéus,
perfeitos para o sol de Luanda.</p> </section> <footer>
<p>Contacta-nos: +244 923 456 789</p> </footer> </body>
</html> ```
```

Ao escreveres esse código, lembra-te de fechar cada tag e de usar aspas corretas nos atributos. O navegador não vai reclamar se tudo estiver bem aninhado, e isso poupa-te dores de cabeça depois.

CSS: a roupa da página

Se o HTML é o esqueleto, o CSS (Cascading Style Sheets) é a roupa que o cobre. Ele decide as cores, as fontes, o espaçamento, e como os elementos se alinham. No caso da tia Maria, poderás escolher um fundo amarelo suave que lembre a areia da praia e uma fonte manuscrita para dar um toque artesanal. O CSS pode ser escrito num ficheiro separado, por exemplo `estilos.css`, e depois ligado ao HTML

com a tag ``<link>``. Assim, manténs a separação entre estrutura e aparência, o que facilita a manutenção.

```
``css body { background-color: #fff8e1; font-family: "Helvetica Neue", Helvetica, Arial, sans-serif; margin: 0; padding: 20px; } header h1 { color: #d2691e; text-align: center; } section img { max-width: 100%; display: block; margin: 0 auto; } footer p { text-align: center; font-size: 0.9em; color: #555; } ``
```

Ao aplicar estas regras, notarás que a página ganha vida. O CSS tem ainda a capacidade de responder a diferentes tamanhos de ecrã – algo essencial se pensas que os clientes da tia Maria vão aceder ao site tanto pelo telemóvel como pelo computador. Usar media queries permite-te ajustar o layout sem mudar o HTML.

JavaScript: a inteligência

O HTML e o CSS entregam-te a cara do site; o JavaScript traz a ação. Com ele, podes validar formulários, criar animações, e até comunicar com servidores para guardar informações. Imagina que queres que o formulário da tia Maria só envie os dados se o campo de telefone estiver preenchido. Um pequeno script pode fazer essa verificação antes de o usuário submeter o pedido.

```
``html <form id="contactForm"> <input type="text" id="phone"
placeholder="Telefone"> <button type="submit">Enviar</button>
</form>
```

```
<script>
document.getElementById('contactForm').addEventListener('submit',
function(e) { var phone =
document.getElementById('phone').value.trim(); if (phone === '') {
e.preventDefault(); alert('Por favor, insere o teu telefone.')} });
</script> ``
```

Além da validação, o JavaScript permite-te buscar dados de um servidor usando a API `fetch`. Assim, poderás, por exemplo, mostrar ao visitante as últimas notícias do mercado local sem precisar recarregar a página. Esta capacidade de interatividade distingue um site estático de um aplicativo web mais sofisticado.

Ferramentas que precisas

Para começar a programar para a web não precisas de um laboratório cheio de equipamentos. Um portátil ou até um telemóvel com um editor de código (como o Visual Studio Code) basta. Instala-te o navegador Chrome ou Firefox – ambos têm ferramentas de desenvolvedor que te permitem inspecionar o HTML e o CSS em tempo real, além de depurar o JavaScript. Se quiseres experimentar rapidamente, o site “codepen.io” permite criar pequenos projetos diretamente no navegador, sem precisar de ficheiros locais. Mas, para um projecto sério como o da tia Maria, cria uma pasta chamada `chapéus-maria`, coloca dentro o `index.html`, o `estilos.css` e um ficheiro `script.js`. Quando tudo estiver a funcionar, poderás subir os ficheiros para um serviço de alojamento gratuito, como o GitHub Pages, e ter o site acessível a qualquer pessoa com a internet.

A prática constante é a chave. Começa por abrir o teu editor, criar um ficheiro HTML simples, e vê-o renderizar no navegador. Depois adiciona uma folha de estilo, altera cores, margens, e observa as mudanças. Por fim, introduz um pequeno script que responde a um clique. Cada passo reforça a tua compreensão e constrói a confiança necessária para projetos maiores.

Quando pensares no futuro, lembra-te de que o desenvolvimento web não termina no front-end. Existem também as linguagens de servidor – como Node.js, que permite escrever JavaScript no lado do servidor – e bases de dados que guardam informações. Mas, antes de

mergulhares nesses mares, dominar o trio HTML, CSS e JavaScript dá-te a base firme para construir tudo o que quiseses.

FAZ ISTO HOJE

Cria uma pasta chamada `primeiro-site` no teu computador.

Dentro dela, cria um ficheiro `index.html` com a estrutura básica de um documento HTML.

Adiciona um título que diga “Bem-vindo ao meu primeiro site”.

Insere um parágrafo que descreva brevemente o teu objetivo ao aprender a programar.

Cria um ficheiro `estilos.css` e liga-o ao `index.html`.

No CSS, define uma cor de fundo suave e altera a cor do texto do parágrafo.

Cria um ficheiro `script.js` e liga-o ao `index.html` antes da tag de fecho ``</body>``.

No JavaScript, adiciona um evento que mostre uma mensagem de alerta quando o utilizador clicar no parágrafo.

Abre o `index.html` no teu navegador e verifica se tudo funciona como esperado.

Se tudo estiver correto, tira uma captura de ecrã e partilha-a com um amigo ou num grupo de estudo.

Projeto simples: cria a tua primeira aplicação

O sol ainda não tinha despontado sobre o bairro da Roça, mas o som de um candongueiro a bater a porta do teu quarto já anunciava o início de mais um dia. Tu acordaste com o cheiro de pão quente da cozinha da mãe, que ainda lhe dava duas colheres de café ao teu lado da mesa. Enquanto esperavas a tua irmã, a Ana, chegar de escola, pegaste no telemóvel e viste a notificação do teu grupo de estudos: “Projeto final: aplicação de lista de compras – prazo amanhã!”. O teu coração disparou. Não há nada mais assustador que ver o prazo a aproximar-se quando ainda não sabes por onde começar. Mas, ao olhar para o teu caderno de anotações, lembraste-te de tudo o que aprendeste nos capítulos anteriores. Esta é a hora de colocar a mão na massa e transformar o que sabes em algo que realmente funciona.

Escolher a ferramenta certa

Depois de montar o teu ambiente de trabalho, a primeira decisão que tens de tomar é a linguagem que vais usar. Para um projeto simples, Python é uma escolha natural: tem sintaxe clara, tem bibliotecas prontas para tudo e corre em qualquer computador, mesmo nos que têm apenas o Windows 10 da escola. Se já instalaste o Python e o editor VS Code, estás pronto. Caso ainda não tenhas o VS Code, abre o teu navegador, vai ao site oficial, baixa a versão para Windows, instala e, depois de abrir, instala a extensão “Python” – isso vai dar-te realce de sintaxe, autocompletar e um terminal integrado.

Lembra-te de que, quando instalaste o Python, escolheste a opção “Add Python to PATH”. Se não fizeste isso, abre o terminal (Ctrl+` no VS Code) e escreve `python --version``. Se aparecer a versão, está tudo bem; se não, volta a instalar e marca a caixa. Não há necessidade de usar gestores de pacotes avançados neste momento – o que precisas já está lá.

Estruturar a aplicação

Mesmo que a aplicação seja simples, pensa em dividir o código em partes distintas. Vai ser mais fácil de entender e de depurar. Para a lista de compras, precisas de três funcionalidades básicas:

1. **Adicionar itens** – o utilizador introduz o nome e a quantidade.
2. **Listar itens** – mostra tudo o que está na lista.
3. **Remover itens** – permite apagar algo que já não precisa.

Cria uma pasta chamada `lista_compras`` no teu desktop. Dentro dela, cria um ficheiro chamado `main.py``. No início do ficheiro, coloca a linha:

```
```python items = [] ```
```

Esta lista vai armazenar todos os itens como dicionários, por exemplo `{'nome': 'Arroz', 'quantidade': 2}``. Mantém a lista vazia até que o utilizador adicione algo.

Agora, define uma função para imprimir o menu. Não precisas de ser sofisticado – um menu de texto funciona perfeitamente:

```
```python def mostrar_menu(): print("\n=== Lista de Compras  
===") print("1 - Adicionar item") print("2 - Listar itens") print("3 -  
Remover item") print("o - Sair") ```
```

A função não devolve nada; apenas escreve no ecrã. O próximo passo é criar as funções de cada ação. Por exemplo, para **adicionar**:

```
```python def adicionar_item(): nome = input("Nome do produto: ").strip() quantidade = input("Quantidade: ").strip() if nome and quantidade.isdigit(): items.append({'nome': nome, 'quantidade': int(quantidade)}) print(f"Adicionado: {quantidade} x {nome}") else: print("Entrada inválida. Tenta novamente.") ```
```

A verificação simples (`quantidade.isdigit()`) impede que o utilizador introduza texto onde se espera um número. Não é preciso ser rigoroso aqui – o objetivo é que o código funcione e que tu te sintas confortável a mudar algo depois.

Para **listar**:

```
```python def listar_itens(): if not items: print("A lista está vazia.") return print("\nItens na lista:") for idx, item in enumerate(items, 1): print(f"{idx}. {item['quantidade']} x {item['nome']}") ```
```

Usamos `enumerate` para mostrar um número ao lado de cada item – isso vai ajudar na hora de remover.

E finalmente, para **remover**:

```
```python def remover_item(): listar_itens() if not items: return escolha = input("Número do item a remover: ").strip() if escolha.isdigit(): idx = int(escolha) - 1 if 0 <= idx < len(items): removido = items.pop(idx) print(f"Removido: {removido['nome']}") else: print("Número fora do intervalo.") else: print("Entrada inválida.") ```
```

A lógica aqui segue o mesmo padrão: verifica se a entrada é um número, converte para índice da lista (lembrando-te de subtrair 1) e, se tudo estiver bem, usa `pop` para retirar o item.

## Controlar o fluxo principal

Depois de teres todas as funções, falta montar o ciclo que vai manter a aplicação a correr até que o utilizador decida sair. No fim do ficheiro `main.py`, coloca:

```
```python def main(): while True: mostrar_menu() opcao = input("Escolhe uma opção: ").strip() if opcao == "1": adicionar_item() elif opcao == "2": listar_itens() elif opcao == "3": remover_item() elif opcao == "o": print("Adeus!") break else: print("Opção inválida. Tenta novamente.") ```
```

E, por fim, garante que o script só corre quando o ficheiro é executado diretamente:

```
```python if __name__ == "__main__": main() ```
```

Agora, abre o terminal integrado do VS Code (Ctrl+`), navega até à pasta `lista_compras` com `cd Desktop\lista_compras` (ou o caminho equivalente) e corre `python main.py`. Vais ver o menu aparecer, pronto para receber as tuas escolhas. Experimenta adicionar alguns itens, listar, remover e, por fim, sair. Se algo não funcionar, respira fundo e verifica a linha onde o erro foi apontado – a maioria dos erros de sintaxe são simples de corrigir.

## Melhorar a aplicação passo a passo

Depois de teres a aplicação a correr, surge a pergunta: “O que mais posso fazer?” Não há necessidade de complicar, mas alguns pequenos aprimoramentos vão dar-te confiança e deixar o teu código mais robusto.

- **Persistir os dados** – Atualmente, quando fechas a aplicação, a lista desaparece. Podes guardar a lista num ficheiro JSON. No início do script, importa `json` e, antes de fechar, escreve

``json.dump(items, open('lista.json', 'w'))``. No arranque, tenta ler o ficheiro com ``json.load``. Se o ficheiro não existir, mantém a lista vazia. Isso ensina-te a lidar com I/O de ficheiros.

- **Validar a quantidade** – Em vez de aceitar qualquer número, verifica se a quantidade é maior que zero. Basta mudar a condição para ``int(quantidade) > 0``.
- **Interface mais amigável** – Se quiseses ir um passo além, experimenta usar a biblioteca ``curses`` para criar um menu que responde a teclas de seta. Não é obrigatório, mas demonstra que Python tem recursos para tudo.

Enquanto trabalhas nestas melhorias, não te esqueças de testar a cada alteração. Um pequeno erro pode fazer o programa travar, mas com o ciclo de depuração que aprendeste nos capítulos anteriores, vais rapidamente descobrir onde está o problema.

## Partilhar o teu trabalho

Chegaste ao fim do teu primeiro projeto. Agora, pensa em como vais mostrar isso a alguém – ao teu professor, a um colega ou até a um futuro empregador. Cria um repositório no GitHub (ou GitLab, caso prefiras). No terminal, inicializa o repositório com ``git init``, adiciona os ficheiros com ``git add .``, faz o primeiro commit (``git commit -m "Primeira versão da lista de compras"``). Depois, cria um repositório online, segue as instruções para ligar o teu diretório local ao remoto e faz ``git push``. O teu código já tem história, tem versões, tem um local onde outras pessoas podem ver o teu esforço.

Ao escrever a descrição do repositório, usa palavras simples e diretas: “Aplicação de lista de compras em Python, permite adicionar, listar e remover itens, guarda os dados em ficheiro JSON”. Não te esqueças de incluir o ficheiro ``README.md`` com instruções de instalação e uso – isso demonstra que sabes comunicar o teu trabalho, algo tão valioso quanto escrever código.

Agora, tu tens uma aplicação funcional, aprendeu a dividir o problema em partes, a tratar erros simples, a guardar dados e a partilhar o teu código. Cada passo reforça a confiança que precisas para avançar a projetos mais complexos. Não te deixes intimidar pelos próximos desafios; lembra-te de que tudo começou num quarto da Roça, com o som do candongueiro e a vontade de fazer algo útil. Continua a praticar, a melhorar e a mostrar o teu trabalho ao mundo.

### **FAZ ISTO HOJE**

Cria uma nova pasta chamada ``lista_compras`` no teu desktop.

Dentro dela, cria o ficheiro ``main.py`` e insere o código apresentado neste capítulo.

Executa o script no terminal do VS Code e testa todas as funcionalidades (adicionar, listar, remover, sair).

Implementa a persistência em ficheiro JSON para que a lista seja guardada ao fechar a aplicação.

Cria um repositório Git, faz o primeiro commit e publica no GitHub com uma breve descrição.

# Depuração e tratamento de erros

---

Mário sentou-se na bancada de madeira do candongueiro da rua 1º de Maio, com o portátil antigo que a mãe lhe deu quando acabou a bolsa de estudos. O sol já começava a aquecer o asfalto, e o som das motos a passar fazia-lhe companhia enquanto ele tentava fazer funcionar o pequeno programa que tinha de entregar para a aula de lógica de programação. O código, que deveria ler um número e dizer se era primo, não passava de um ecrã preto com a mensagem “ZeroDivisionError: division by zero”. Mário suspirou, abriu o editor e começou a procurar o ponto onde tudo tinha falhado. Foi ali, entre o ruído da cidade e o brilho da tela, que aprendeu que depurar não é só corrigir erros: é entender o que o teu código pensa.

## Quando o erro fala

A primeira coisa que um programador tem de aceitar é que o código nunca vai fazer exatamente o que esperamos à primeira tentativa. Cada exceção, cada mensagem de erro, é um sinal que o programa lhe está a dizer algo. Em vez de fugir a essa conversa, abraça-a.

```
```python def é_primo(n): for i in range(2, n): if n % i == 0: return False return True ```
```

Se correremos `é_primo(0)`, o laço tenta iterar de 2 até 0, o que gera um range vazio e devolve `True`. O erro não aparece, mas o resultado está errado. O primeiro passo da depuração é reproduzir o erro com

dados que o provoquem. Mário tentou ``é_primo(o)`` e percebeu que o programa dizia que o era primo – algo que claramente não faz sentido. Assim, o problema estava no próprio algoritmo, não só na exceção.

Ferramentas de inspeção rápida

Print, o teu primeiro debugger

A forma mais simples de observar o que acontece dentro de uma função é imprimir valores intermédios. Não é preciso nada sofisticado; basta colocar ``print`` nos pontos críticos.

```
```python def é_primo(n): print(f"Entrada: {n}") for i in range(2, n): print(f"Testando divisor {i}") if n % i == 0: print(f"{i} divide {n}") return False return True ```
```

Ao correr ``é_primo(9)``, o terminal mostrará cada divisor testado, permitindo perceber que o algoritmo verifica 3 e descobre que 3 divide 9. Se o programa ainda não estiver a comportar-se como esperas, o ``print`` revela onde o fluxo diverge.

### O depurador integrado

A maioria dos IDEs (VS Code, PyCharm, Eclipse) tem um depurador que permite colocar “breakpoints”. Quando o código atinge um breakpoint, a execução pára e podes inspeccionar variáveis, avançar linha a linha e mudar valores em tempo real. Mário, depois de cansar os ``print``, decidiu abrir o VS Code, definiu um ponto de paragem na linha do ``for`` e, ao avançar passo a passo, viu que o ``range`` estava a gerar valores inesperados quando ``n`` era negativo. Essa descoberta salvou-o de um bug que teria sido difícil de achar só com ``print``.

### Utilizar o traceback

Quando uma exceção é levantada, o Python devolve um traceback que indica a cadeia de chamadas que levou ao erro. Lê-lo com atenção. No caso de Mário, a mensagem era:

```
``` ZeroDivisionError: division by zero File "primo.py", line 5, in
é_primo if n % i == 0: ```
```

O traceback mostrou que o erro ocorria na linha do módulo, não num ponto distante do código. O facto de a exceção ser “division by zero” indicava que `i` tinha sido 0 – algo que nunca deveria acontecer num `range(2, n)`. Ao analisar o código, percebeu que tinha usado `range(0, n)` por engano numa versão anterior.

Tratamento de exceções: não deixes o programa falhar

Se um erro pode acontecer, trata-o antes que o programa se encerre abruptamente. Em Python, usa-se `try ... except`.

```
```python def dividir(a, b): try: return a / b except
ZeroDivisionError: print("Não podes dividir por zero.") return None
```
```

Ao chamar `dividir(5, 0)`, o programa não explode; devolve `None` e informa o utilizador. Mário adicionou este padrão ao seu projecto de cálculo de média, porque um aluno podia introduzir “0” como nota de teste e, sem o `try`, o programa teria terminado antes de imprimir a média da turma.

Capturar múltiplas exceções

Nem todos os erros são de divisão por zero. Um `ValueError` pode surgir quando tentas converter texto em número:

```
```python def ler_int(texto): try: return int(texto) except
ValueError: print(f'"{texto}" não é um número inteiro.') return None
```

```

Ao combinar `try ... except` com `else` e `finally`, ganhases controlo total sobre o fluxo:

```
```python try: numero = ler_int(input("Escreve um número: "))
resultado = dividir(10, numero) except Exception as e:
print(f'Ocorreu um erro inesperado: {e}') else: print(f'O resultado é
{resultado}') finally: print("Operação concluída.") ```
```

O bloco `else` só corre se nada falhar; o `finally` executa-se sempre, útil para fechar ficheiros ou libertar recursos.

## Logging: guardar o que aconteceu

Depurar durante o desenvolvimento é diferente de depurar em produção. Quando o teu código já está a ser usado por outras pessoas, não podes ficar a imprimir tudo no ecrã. Em vez disso, usa o módulo `logging`.

```
```python import logging

logging.basicConfig(filename='app.log', level=logging.INFO,
format='%(asctime)s %(levelname)s: %(message)s')

def é_primo(n): logging.info(f'Verificando se {n} é primo') if n < 2:
logging.warning(f'Número {n} inválido para teste de primalidade")
return False for i in range(2, n): if n % i == 0: logging.info(f'{i} divide
{n}') return False return True ```
```

Agora, ao correr o programa, tudo o que acontece fica guardado em `app.log`. Se um utilizador reportar um erro, podes abrir o ficheiro de log e ver exatamente que valores foram processados. Mário implementou o logging no seu projecto final e, quando um colega

encontrou um “IndexError”, o log revelou que ele tinha introduzido um número negativo, algo que o programa não estava a validar.

Estratégias de depuração avançada

Testes unitários

Escrever pequenos testes que verificam funções isoladas impede que erros se espalhem. Usa o módulo `unittest` ou `pytest`. Um teste para a função `é\_primo` pode ser:

```
```python import unittest from primo import é_primo

class TestPrimo(unittest.TestCase):
 def test_primos(self):
 self.assertTrue(é_primo(2))
 self.assertTrue(é_primo(13))
 def test_nao_primos(self):
 self.assertFalse(é_primo(4))
 self.assertFalse(é_primo(0))

if __name__ == '__main__':
 unittest.main()```
```

Executar o conjunto de testes antes de entregar o projecto dá confiança de que as alterações não romperam funcionalidades existentes.

### ### Refatoração e isolamento

Se uma função está demasiado grande ou tem muitas responsabilidades, separa-a em partes menores. Cada parte pode ser testada isoladamente, facilitando a localização de bugs. Por exemplo, dividir a verificação de primalidade em duas funções – uma que gera os divisores e outra que verifica a divisibilidade – permite testar cada passo sem depender do outro.

### ### Revisão de código

Um par de olhos frescos costuma encontrar falhas que a tua mente já habituou a ignorar. Partilha o teu código com um colega, num grupo de estudo ou num fórum de desenvolvedores. O feedback pode apontar um caso de canto que ainda não tinhas considerado.

## Quando a depuração falha: aceitar limites

Nem todo bug tem solução imediata. Às vezes, o problema está na própria especificação – o que o utilizador espera não está bem definido. Nesses casos, volta à análise de requisitos, pergunta ao cliente ou ao professor qual o comportamento desejado. Mário, ao confrontar um erro que surgia apenas quando a entrada era uma string vazia, descobriu que o enunciado da tarefa não especificava o que fazer nesse cenário. Depois de esclarecer a regra, o código ficou completo.

### FAZ ISTO HOJE

Abre o teu editor e cria um ficheiro chamado ``debug_exemplo.py``.

Copia para o ficheiro o seguinte código incompleto:

```
```python
def soma_lista(nums):
    total = 0
    for n in nums:
        total += n
    return total
print(soma_lista([1, 2, 'três', 4]))
```
```

Executa o script e observa a mensagem de erro que aparece.

Adiciona prints dentro da função para mostrar o valor de `n` a cada iteração.

Usa um bloco `try ... except` para capturar o erro que ocorre quando `n` não é um número.

Regista a informação de erro num ficheiro `erro.log` usando o módulo `logging`.

Cria um pequeno teste unitário que verifica se `soma_lista([1, 2, 3])` devolve `6` e se `soma_lista([])` devolve `0`.

## CAPÍTULO 11

# Controlo de versão com Git

---

O teu relógio marcava 07:45 quando o candongueiro do bairro Cazenga chegou com um carregamento de kwanzas e de um velho portátil, que o teu tio, o Zé, tinha trazido de Lisboa. O ecrã ainda mostrava a tela de login do Windows, mas o teu olhar ficou preso naquele pequeno ícone azul – o logo do Git. “É assim que guardam o código?”, perguntou-te o Zé, a dizer-me que queria aprender a desenvolver uma aplicação de controlo de estoque para a loja da irmã. Sentei-me ao teu lado, abraçando o teclado, e percebi que aquele momento seria o primeiro passo para domar o caos dos projetos de código.

## Por que o Git mudou o jogo

A maioria dos programadores que ainda não usou Git sente que o código vive numa caixa de papelão: cada alteração é feita sobre a anterior, e se algo falha, volta-se ao ponto de partida como se fosse impossível avançar. O Git introduz um registo completo das tuas mudanças, permitindo que voltes no tempo, experimentes ramificações diferentes e, sobretudo, trabalhes em equipa sem medo de sobrescrever o trabalho dos outros. Não é só uma ferramenta, é um diário que te acompanha ao longo do percurso de desenvolvimento. Se conseguires dominar o Git, ganharás confiança para arriscar, para refazer e para criar soluções mais robustas – algo que nenhum manual de lógica de programação pode oferecer por si só.

## O teu primeiro repositório: criando raízes

Começar é tão simples como abrir o terminal e escrever:

```
``` git init ```
```

Este comando cria um diretório oculto chamado ``.git`` onde o Git vai guardar todo o histórico. Imagina que estás a plantar uma árvore no quintal da tua casa: o solo é o teu projeto, e o ``.git`` são as raízes que vão sustentar tudo o que crescer. A partir daí, adicionas os arquivos que queres versionar com:

```
``` git add . ```
```

Não precisas de explicar-te demais; o ponto significa “todos os ficheiros”. Depois, registas a primeira fotografia do teu código:

```
``` git commit -m "Primeiro commit – estrutura inicial" ```
```

O `-m`` permite colocar uma mensagem clara; pensa em algo que te recorde o que fizeste, como se fosse uma nota no teu caderno de aulas.

Ramificações: o campo de treino

Quando construístes a aplicação de estoque, rapidamente percebeste que precisavas de duas funcionalidades distintas: uma para registar entradas e outra para gerar relatórios. Em vez de misturar tudo num único fluxo, criaste duas ramificações:

```
``` git checkout -b entrada ```
```

e

```
``` git checkout -b relatorio ```
```

A palavra-chave ``-b`` cria a ramificação e muda-te para ela. Cada ramo funciona como um campo de treino separado; as alterações que

fizeres em `entrada` não afetam `relatorio` até que decides combinar os dois. Enquanto trabalhas na ramificação de entrada, podes fazer commits que só dizem respeito à inserção de produtos, e o mesmo acontece na ramificação de relatórios. Essa separação evita conflitos e permite que tu e o teu colega de equipa (ou o Zé) trabalhem simultaneamente, cada um no seu terreno.

Unindo forças: o merge sem drama

Chegou a hora de juntar as duas ramificações. Primeiro, volta à ramificação principal (geralmente chamada `master` ou `main`):

```
``` git checkout main ```
```

Depois, puxa as alterações da ramificação `entrada`:

```
``` git merge entrada ```
```

Se tudo correr bem, o Git integrará as mudanças automaticamente. Caso exista alguma sobreposição – por exemplo, ambos alteraram o mesmo ficheiro de configuração – o Git sinaliza um conflito, pedindo que escolhas a versão correta. Esse processo pode parecer complicado, mas é uma oportunidade para reveres o código e garantir que nada se perde. Resolve o conflito, guarda o ficheiro, e finaliza o merge com:

```
``` git commit -m "Merge da ramificação entrada" ```
```

Repete o mesmo para a ramificação `relatorio`. No final, a ramificação principal contém as duas funcionalidades integradas, prontas para serem testadas em conjunto.

## Partindo para o mundo: repositórios remotos

Até aqui trabalhaste apenas no teu computador. Para partilhar o teu trabalho, precisas de um repositório remoto, como o GitHub ou o

GitLab. Cria uma conta, cria um novo repositório vazio e copia o URL. Depois, liga o teu repositório local ao remoto:

```
``` git remote add origin https://github.com/teu-usuario/estoque.git ```
```

O nome `origin` é apenas uma convenção, mas ajuda a identificar o destino principal. Agora podes enviar as tuas alterações para a nuvem:

```
``` git push -u origin main ```
```

O `-u` define a ramificação padrão para futuros `push`. Cada vez que fizeres um commit, basta correr `git push` para atualizar o repositório remoto. Assim, o Zé poderá clonar o projeto no portátil dele, revisar o código e contribuir com novas funcionalidades sem medo de sobrescrever nada. Esta colaboração é a própria essência do Git: um registo partilhado que mantém a integridade do projeto, mesmo quando há várias mãos a trabalhar.

## Boas práticas que não são regras rígidas

Não há necessidade de transformar o teu fluxo de trabalho num ritual monótono; porém, algumas atitudes ajudam a tornar o Git mais útil. Usa mensagens de commit curtas mas descritivas – pensa num título de notícia que resume a alteração. Evita fazer commits gigantes com múltiplas funcionalidades, pois isso dificulta a revisão de código. Quando criares uma ramificação, dá-lhe um nome que reflita a tarefa, como `correcao-bug-123` ou `nova-funcionalidade-pesquisa`. Se te sentires perdido, o comando `git status` mostra onde estás e o que ainda não foi adicionado ao próximo commit. Por fim, lembra-te de fazer pull regularmente antes de iniciar o teu trabalho: isso garante que tens a versão mais atual do código remoto e reduz a probabilidade de conflitos.

## O Git como aliado na depuração

Um dos momentos mais frustrantes na programação é descobrir que um bug surgiu depois de uma série de alterações. Com o Git, podes retroceder a um commit anterior e testar o estado do código naquele ponto. Por exemplo:

```
``` git checkout <hash-do-commit> ```
```

O `<hash-do-commit>` é a sequência curta que identifica cada commit. Ao voltar a esse ponto, executas a tua aplicação e verificas se o bug persiste. Se desaparecer, sabes que a mudança que introduziste depois daquele commit é a culpada. Essa capacidade de “viajar no tempo” faz do Git um poderoso aliado na depuração, pois permite isolar problemas sem ter de reescrever todo o código.

Mantendo o teu portefólio em forma

Quando chega a hora de mostrar o teu trabalho a um potencial empregador, o Git pode ser o teu cartão de visita. Um repositório bem organizado, com commits claros e ramificações bem nomeadas, demonstra não só competência técnica, mas também disciplina e capacidade de trabalhar em equipa. Inclui um ficheiro `README.md` que explique o propósito da aplicação, as tecnologias usadas e como a instalar. Se quiseres, adiciona um `CHANGELOG.md` que documente as versões e as principais alterações. Esses pequenos detalhes elevam o teu portefólio de um simples conjunto de ficheiros a um projeto profissional que reflete quem és como programador.

🔧 FAZ ISTO HOJE

Cria um diretório novo chamado `gestao-estoque` e entra nele.

Inicializa um repositório Git dentro do diretório.

Cria um ficheiro `README.md` com uma breve descrição da aplicação.

Faz o primeiro commit adicionando o `README.md`.

Cria duas ramificações chamadas `entrada` e `relatorio`.

Em `entrada`, adiciona um ficheiro `entrada.py` com um esqueleto de função que registe um produto.

Em `relatorio`, adiciona um ficheiro `relatorio.py` com uma função que liste os produtos.

Faz commit em cada ramificação com mensagens descritivas.

Volta à ramificação principal e faz merge das duas ramificações, resolvendo eventuais conflitos.

Cria um repositório remoto no GitHub (pode ser privado) e liga-o ao teu repositório local.

Faz push da ramificação principal para o repositório remoto.

Verifica no GitHub que os ficheiros e os commits aparecem corretamente.

Boas práticas e legibilidade

O Pedro, de 17 anos, estava sentado à mesa da sua casa em Talatona, rodeado por cadernos de matemática, um copo de sumo de maracujá e o portátil que lhe tinha sido presenteado no aniversário. Naquela tarde, o professor de informática da escola secundária de Talatona lhe pediu para melhorar um pequeno programa que calculava a média das notas de cada disciplina. O código funcionava, mas tinha três linhas de 70 caracteres, nomes de variáveis como ``x1``, ``x2``, ``x3`` e nenhum comentário. Quando o Pedro mostrou o programa ao colega de classe, este não conseguiu perceber o que cada parte fazia. Sentiu-se frustrado e percebeu que, se o código não fosse legível, jamais seria útil para ninguém – nem para ele próprio. Foi então que decidiu aprender as boas práticas que transformam código confuso em algo que qualquer pessoa pode ler e manter.

Porquê a legibilidade conta mais que a rapidez

Um programa pode correr a mil por cento da velocidade esperada, mas se ninguém o entender, as empresas, as equipas de projecto e tu próprio, quando volta a olhar ao código depois de alguns meses, ficarão perdidos. A legibilidade reduz o custo de manutenção, facilita a deteção de bugs e acelera a colaboração. Quando trabalhas num projeto de grupo, como o de um site para a associação da tua freguesia, cada membro traz o seu estilo. Se todos seguirem um conjunto de regras simples – nomes claros, indentação consistente,

comentários úteis – o projecto avança mais rapidamente porque ninguém tem de “desenterrar” o que está escrito.

Nomes que falam por si

A primeira prática que muda a percepção do teu código são os nomes das variáveis e das funções. Em vez de ``a``, ``b`` ou ``x1``, pensa no que aquela peça de informação representa. Se guardas a nota de Matemática, chama-a ``nota_matematica``; se calculas a média, cria uma função ``calcular_media``. Usa o *snake_case* (palavras separadas por underscores) ou o *camelCase*, mas mantém a consistência. Um nome bem escolhido elimina a necessidade de comentário adicional, porque o próprio nome já explica a intenção.

Exemplo antes:

```
python x1 = 85 x2 = 73 x3 = 90 y = (x1 + x2 + x3) / 3
```

Exemplo depois:

```
python nota_matematica = 85 nota_portugues = 73 nota_ciencias = 90 media = (nota_matematica + nota_portugues + nota_ciencias) / 3
```

A diferença é óbvia. Quando lês o segundo bloco, sabes imediatamente que estás a trabalhar com notas e que a última linha devolve a média. Se o teu código fosse parte de um sistema de registo de notas nacional, essa clareza pouparia horas de explicação a colegas de outras províncias.

Indentação e espaçamento: a estrutura visual do teu código

A linguagem Python, a que usamos neste livro, depende da indentação para definir blocos de código. Um erro de espaçamento pode gerar exceções difíceis de depurar. Mas, para além da exigência técnica, a indentação cria um mapa visual que o leitor segue com o olho.

Mantém a mesma quantidade de espaços (quatro é o padrão) em todos os níveis e evita misturar tabs e espaços. Se utilizares um editor como VS Code ou PyCharm, activa a opção “format on save” – assim o teu código será formatado automaticamente.

Quanto ao espaçamento interno, adiciona um espaço antes e depois dos operadores (`+', '-, '*, '/, '=`) para melhorar a legibilidade. Por exemplo, ``media=(nota1+nota2)/2`` torna-se ``media = (nota1 + nota2) / 2``. Essa pequena mudança faz com que o código pareça mais “respirado”, permitindo que o cérebro processe cada operação de forma independente.

Comentários que realmente ajudam

Um comentário não deve ser um bloco de texto que repete o que já está no código. Em vez disso, serve para esclarecer *porquê* algo foi feito de determinada maneira, ou para indicar limitações e decisões de design. Usa comentários de linha curta (``#``) acima do bloco que explicam a intenção, e guarda comentários extensos para documentação externa.

Bom comentário: ````python # Calcula a média ponderada das notas, considerando que Matemática tem peso 2 def calcular_media_ponderada(nota_matematica, nota_portugues, nota_ciencias): peso_total = 2 + 1 + 1 soma_ponderada = nota_matematica * 2 + nota_portugues + nota_ciencias return soma_ponderada / peso_total ````

Mau comentário: ````python # Função que calcula a média def calcular_media_ponderada(a, b, c): # soma dos valores # peso total = 4 return (a*2 + b + c) / 4 ````

No segundo exemplo, o comentário “soma dos valores” não acrescenta nada que já não se veja no código. O primeiro, ao indicar o

peso da disciplina, esclarece a lógica por trás do cálculo, ajudando quem for ler a perceber rapidamente a regra de negócio.

Estrutura modular e reutilização

Dividir o código em funções pequenas e coesas facilita a leitura e a reutilização. Cada função deve fazer *uma única coisa* e ter um nome que reflita essa ação. Se precisas de validar notas antes de calcular a média, cria uma função `validar\_nota` que devolve `True` ou `False`. Assim, a função principal fica limpa e a lógica de validação pode ser testada isoladamente.

```
```python def validar_nota(nota): return 0 <= nota <= 100
```

```
def calcular_media(notas): notas_validas = [n for n in notas if
validar_nota(n)] if not notas_validas: raise ValueError('Nenhuma
nota válida') return sum(notas_validas) / len(notas_validas) ```
```

Além de melhorar a legibilidade, a modularidade permite que, no futuro, alteres a forma de validação (por exemplo, introduzindo um novo intervalo de aprovação) sem tocar na lógica de cálculo da média. Essa separação de responsabilidades é uma das bases das boas práticas de programação.

## Testes simples que reforçam a confiança

Mesmo que ainda não tenhas abordado testes automatizados, escrever alguns *asserts* no final do teu script ajuda a garantir que as funções se comportam como esperado. Não é necessário um framework complexo; basta verificar alguns casos típicos.

```
```python assert validar_nota(50) is True assert validar_nota(-5)  
is False assert round(calcular_media([80, 90, 100]), 2) == 90.0 ```
```

Se alguma dessas afirmações falhar, o Python levantará uma exceção imediatamente, indicando que algo está errado. Esse hábito cria uma camada extra de segurança antes de partires para projetos maiores, onde a confiança no código é ainda mais crítica.

Documentação mínima mas suficiente

Além dos comentários internos, mantém um ficheiro `README.md` no teu projecto, explicando o propósito, as dependências e como executar o programa. Uma documentação curta, mas clara, poupa tempo a quem quiser usar ou contribuir para o teu código. No README, inclui um exemplo de entrada e saída, de forma a que quem o abra saiba o que esperar.

Boas práticas no controlo de versão

Embora tenhas um capítulo dedicado ao Git, vale a pena reforçar que commits pequenos e bem descritos são parte da legibilidade. Cada commit deve refletir uma mudança lógica – “Adiciona validação de notas” ou “Refatora cálculo da média para usar peso”. Evita commits gigantes que misturam várias alterações; eles tornam o histórico confuso e dificultam o revert de um erro específico.

Conclusão prática

A legibilidade não é um luxo; é a base que sustenta todo o resto do teu desenvolvimento. Quando seguires estas práticas – nomes claros, indentação consistente, comentários úteis, modularidade e testes simples – o teu código falará por si. Os teus colegas, futuros empregadores e tu próprio reconhecerás imediatamente a qualidade do teu trabalho, e cada minuto que investires agora economizar-te-á horas de depuração mais tarde.

FAZ ISTO HOJE

Abre o teu editor e cria um ficheiro chamado ``media_notas.py``.

Define três variáveis com nomes descritivos para notas de Matemática, Português e Ciências.

Escreve uma função ``validar_nota`` que verifica se a nota está entre 0 e 100.

Cria uma função ``calcular_media_ponderada`` que usa pesos 2 para Matemática e 1 para as demais disciplinas.

Adiciona comentários que expliquem o porquê dos pesos.

No fim do ficheiro, inclui três ``assert`` que testem a validação e o cálculo da média.

Executa o script e verifica que não surgem erros.

Preparar o teu portefólio

O sol já começava a queimar o asfalto do bairro da Alma, quando o Tiago, de 19 anos, saiu da escola do Candongueiro e dirigiu-se ao mercado do Kilamba. Entre as bancas de frutas, o som das negociações e o cheiro de peixe frito, ele encontrou o seu primo, o Rafael, que acabara de arrumar o seu primeiro emprego como programador numa startup de fintech. “Olha, Tiago, se quiseres entrar nesta área, tens de ter algo que mostre o que sabes fazer”, disse o primo, apontando para o portátil aberto sobre a mesa. Tiago ficou a observar a tela, onde um pequeno site mostrava gráficos de investimentos. Naquele instante, sentiu-se a primeira faísca: o portefólio seria a ponte entre o seu futuro e a realidade do mercado.

O que o teu portefólio tem de mostrar

Um portefólio não é um simples repositório de código; é a tua carta de apresentação, a prova viva de que consegues transformar ideias em soluções. Quando um recrutador abre o teu GitHub ou o teu site, ele procura três coisas: capacidade técnica, pensamento de resolução de problemas e capacidade de comunicar. Por isso, não basta colocar projetos aleatórios; tem de escolher trabalhos que revelem a tua jornada e o teu estilo. Se tens experiência com Python, mostra um script que automatiza uma tarefa do dia-a-dia, como recolher preços no mercado para comparar com o preço do pão. Se já experimentaste JavaScript, cria uma página que interaja com o utilizador, talvez um pequeno “quiz” sobre a história de Angola. Cada projeto deve

responder a uma pergunta: “Que problema eu resolvi?” e “Como o fiz?”. Quando conseguires responder a essas questões de forma clara, estarás a construir um portefólio que realmente fala por ti.

Selecionar os projetos certos

Não te deixes enganar pela quantidade. É melhor ter três projetos bem elaborados do que dez que mal chegam a ser funcionais. Lembra-te de quando fizeste o teu primeiro “Hello World” e ficaste satisfeito por teres conseguido compilar o código; agora, o objetivo é avançar para algo que mostre que sabes estruturar, testar e documentar. Pensa nos projetos que já fizeste nas aulas: a aplicação de lista de tarefas, o mini-jogo de adivinhação, o pequeno site de notícias. Escolhe aqueles que melhor representem as competências que mais valorizam as empresas que te interessam – por exemplo, se pretendes trabalhar em e-commerce, inclui um projecto que manipule carrinhos de compra e pagamentos simulados. Se ainda não tens um projeto “real”, cria um que resolva um problema do teu bairro: um sistema de registo de doações para a igreja da tua rua, ou um calendário de eventos comunitários. A autenticidade acrescenta valor, porque demonstra que estás atento ao que acontece à tua volta e que usas a programação para melhorar a realidade.

Documentação que convence

Um código sem explicação pode parecer arrogante ou, pior, incompreensível. Quando alguém abre o teu repositório, a primeira coisa que deve encontrar é um README bem escrito. Começa por explicar brevemente a ideia do projecto, depois descreve as tecnologias usadas e, sobretudo, indica como o executar. Se o teu projecto requer a instalação de dependências, escreve instruções claras, usando comandos que funcionam tanto no Windows como no

Linux, para não afastar quem não conhece o teu ambiente. Não te esqueças de incluir exemplos de uso, capturas de ecrã e, se possível, um link para a aplicação em execução (por exemplo, no Vercel ou no Netlify). Quando descreves as decisões que tomaste – por que optaste por uma lista em vez de um dicionário, ou por que usaste um padrão MVC –, mostras pensamento crítico e maturidade. Essa camada de explicação transforma o teu código num relato que qualquer leitor consegue seguir, mesmo que não saiba tudo sobre a linguagem.

Ferramentas e alojamento

Hoje em dia, o teu portefólio pode viver em dois mundos: o código no GitHub e a aplicação num serviço de alojamento. O GitHub oferece controlo de versão, colaboração e um local onde os recrutadores podem ver a tua atividade. Aprende a usar branches e pull requests, e mantém-te fiel às boas práticas que aprendeste nos capítulos anteriores. Para a apresentação visual, cria um site estático que mostre os teus projectos, com uma página “Sobre Mim” que inclua a tua formação, interesses e contacto. Usa um domínio próprio, se possível; um endereço como tiago.dev soa mais profissional do que um link genérico. Plataformas como GitHub Pages, Vercel ou Netlify permitem publicar rapidamente e sem custos. Ao escolheres a tecnologia para o teu site, pensa também na performance: imagens otimizadas, tempos de carregamento curtos e uma navegação simples fazem a diferença. Um portefólio que carrega rápido transmite a mensagem de que te importas com a experiência do utilizador – algo essencial para quem trabalha com desenvolvimento web.

Mantém o teu portefólio vivo

Um portefólio não é um projecto estático; tem de evoluir à medida que tu aprendes e experimentas. Reserva algum tempo a cada mês

para rever os teus projectos, atualizar dependências e melhorar a documentação. Se concluíste um curso de React ou aprendeste a usar Docker, inclui um exemplo que mostre essas novas competências. Não tenhas medo de remover projetos antigos que já não representam o teu nível atual; um portefólio desatualizado pode dar a impressão de que não estás a evoluir. Também é útil receber feedback – partilha o teu site com amigos, colegas ou mentores e pergunta o que eles acham que pode ser melhorado. Cada crítica construtiva é uma oportunidade para refinar a tua apresentação. Lembra-te de que o teu portefólio é a tua vitrine profissional; mantê-la limpa, organizada e atualizada é tão importante quanto escrever código limpo.

FAZ ISTO HOJE

Cria um repositório no GitHub chamado "portefolio-tiago" e adiciona um README que descreva quem és, quais as tuas competências e a motivação para o portefólio.

Desenvolve uma aplicação web simples (por exemplo, um calendário de eventos comunitários) usando HTML, CSS e JavaScript, e publica-a em GitHub Pages.

Documenta o projecto no README, incluindo instruções de instalação, captura de ecrã e um link para a aplicação em funcionamento.

Partilha o link do teu portefólio com três pessoas de confiança e pede-lhes feedback específico sobre clareza, usabilidade e conteúdo.

Implementa pelo menos uma das sugestões recebidas e actualiza o repositório com a nova versão.

CAPÍTULO 14

Plano de 90 dias para tornar-te programador

Era uma manhã quente no bairro de Viana, quando o candongueiro do Zé, o “Boi”, chegou à esquina da Rua do Ferro com o Mercado dos Talentos. O motor roncava, a caixa de som ainda ecoava o último hit de Kuami, e eu, com o portátil já aberto na mão, esperava o seu “ponto” para comprar um garrafa de água. Quando o Boi parou, o velho Mário, que vendia pastel de carne na esquina, disse-me: «Já viste a nova app que o teu primo está a criar? O patrão da empresa de telecomunicações ficou a notar. Se quiseres aprender a programar, tem de ser rápido, tem de ser prático, tem de ser real». Sentei-me no capô, olhei para o ecrã e percebi que, ali, naquele instante, a oportunidade estava a bater à porta. Não havia mais tempo para teorias vazias; precisava de um plano que me levasse de zero a programador em noventa dias.

Definir metas que realmente importam

Antes de colocar a mão no teclado, decide-te que tipo de programador queres ser. Não adianta dizer «vou ser desenvolvedor full-stack», se a tua primeira aplicação vai ser uma calculadora de juros para a família. Pergunta-te: «Qual o problema que quero resolver? Qual a empresa ou a área que me atrai?». Quando trago a resposta ao papel – ou ao teu bloco de notas – transformas a ideia abstrata num objetivo concreto, como «construir uma aplicação web que mostre os preços dos produtos do Mercado dos Talentos em tempo real». Essa meta

tem três vantagens: é mensurável, tem um prazo (noventa dias) e tem relevância pessoal.

Ao definir metas, não te enganes com o que os outros dizem. Se o teu primo já tem experiência em Python, mas tu preferes JavaScript porque te intriga a interação com o DOM, segue essa escolha. O caminho que traças tem de ser teu, não uma cópia de alguém. Cada decisão que tomas neste estágio define a rota que vais percorrer nos próximos três meses.

Dividir o percurso em blocos de aprendizagem

Noventa dias podem parecer pouco, mas quando se partem em blocos de duas a três semanas torna-se manejável. Imagina o calendário como um tabuleiro de damas: cada quadrado representa uma fase que, ao ser concluída, abre caminho para a próxima.

1. Fundamentos (dias 1-15) – Concentra-te nas bases: variáveis, tipos, operadores e estruturas de controlo. Faz pequenos scripts que imprimam mensagens, que calculem descontos ou que validem dados de um formulário. O objetivo aqui é sentir-te confortável a escrever código que funciona.

2. Manipulação de dados (dias 16-30) – Aprende a trabalhar com listas e dicionários. Cria um pequeno programa que receba o nome dos produtos do mercado e guarde-os num dicionário com o preço e a quantidade em stock. Experimenta ordenar, filtrar e agrupar os dados. Esta fase dá-te a base para projetos mais complexos.

3. Desenvolvimento web (dias 31-45) – Introduz-te ao HTML, CSS e ao JavaScript no navegador. Faz um site estático que mostre a lista de produtos que criaste na fase anterior. Usa fetch para fazer pedidos a um “mock API” (podes usar jsonplaceholder ou criar um

ficheiro JSON local). Concentra-te em perceber como o cliente e o servidor trocam informação.

4. Projeto prático (dias 46-75) – Junta tudo num projecto real: uma aplicação que, a partir de um formulário, permite ao utilizador inserir novos produtos, editar os existentes e remover aqueles que já não estão disponíveis. Implementa o armazenamento em local-storage para que os dados persistam entre sessões. Esta é a tua “primeira aplicação” que demonstra capacidade de ponta-a-ponta.

5. Polimento e preparação para o mercado (dias 76-90) – Refina o código, adiciona tratamento de erros, documenta as funções e cria um pequeno portefólio online (pode ser um GitHub Pages). Aprende a usar Git para versionar o teu projecto, a fazer commits claros e a abrir um pull request para ti mesmo. No fim deste bloco, tens um produto acabado que podes mostrar ao patrão da empresa de telecomunicações que Mário mencionou.

Dividir o percurso desta forma evita a sensação de sobrecarga e permite medir o progresso a cada quinzena. Se alguma fase não correr como esperas, ajusta-a antes de avançar; não deixes que um bloco atrasado arraste o plano inteiro.

Rotina diária de estudo: ritmo que funciona para ti

A maioria dos estudantes subestima o poder de uma rotina curta mas consistente. Em vez de reservar “três horas de estudo” num fim de semana, procura encaixar sessões de 45 a 60 minutos todos os dias. O teu cérebro tem mais facilidade em consolidar informação quando o input é regular.

Manhã – Logo após o pequeno-almoço, dedica-te a ler teoria ou a assistir a um vídeo curto (máximo 20 minutos). Faz anotações à mão; o ato de escrever ajuda a fixar conceitos. Por exemplo, se estás a

estudar loops, escreve um pequeno resumo com exemplos de ``for`` e ``while``.

Tarde – Depois da aula ou do trabalho, faz um exercício prático. Não basta copiar código; adapta-o ao teu contexto. Se estás a trabalhar num projecto de lista de produtos, tenta mudar a lógica de ordenação para criar um ranking dos mais vendidos.

Noite – Reserva-te 10 minutos para rever o que fizeste durante o dia e anotar dúvidas. Usa um bloco de notas ou uma aplicação de gestão de tarefas (como Trello ou Notion) para marcar o que concluíste e o que ainda falta. Esta revisão diária reforça a memória e te permite identificar rapidamente os pontos fracos.

A chave é a disciplina, mas também a flexibilidade. Se um dia não consegues estudar à tarde porque tens um compromisso, compensa-te com uma sessão extra à noite. O plano não é rígido como uma maratona, mas tem a elasticidade de um bambu que se dobra sem quebrar.

Ferramentas de acompanhamento e medição de progresso

Para não te perderes no caminho, escolhe ferramentas que te ajudem a medir o teu avanço sem criar burocracia. Uma boa prática é manter um **diário de código**: um ficheiro ``progress.md`` no teu repositório onde registas, dia a dia, o que aprendeu, o que implementaste e as dificuldades encontradas. Além de servir como documentação, este diário será um recurso valioso quando quiseres refletir sobre o teu percurso.

Outra ferramenta essencial é o **Git**. Cada bloco de aprendizagem pode ser um branch separado (``fundamentals``, ``data-manipulation``, etc.). Faz commits ao final de cada sessão de estudo; assim, consegues

voltar a versões anteriores se algo falhar. Quando chegares ao final do ciclo de 90 dias, poderás criar um **release** que consolida todo o trabalho.

Finalmente, usa um **timer Pomodoro**. Trabalhar em blocos de 25 minutos focados, seguidos por pausas curtas, melhora a concentração. Se sentires que o teu foco escorrega, experimenta sessões de 15 minutos; o importante é manter a qualidade da prática.

Ajustar e validar o teu plano

Nenhum plano nasce perfeito. Depois de duas semanas, faz uma pausa de 30 minutos para avaliar: consegues cumprir as metas diárias? O conteúdo que estudaste está a gerar resultados concretos? Se a resposta for negativa, revisita o teu cronograma. Talvez precises de mais tempo nas bases ou menos nas ferramentas avançadas.

Um método simples de validação é o **testes de aplicação real**. Convida um amigo ou um familiar a usar a tua aplicação de lista de produtos. Pergunta-lhe se a interface é intuitiva, se os resultados são corretos e se algo falta. O feedback externo oferece uma perspetiva que o teu próprio cérebro pode ignorar.

Se, ao fim do terceiro mês, ainda não tens um projecto funcional, reconsidera a complexidade da tua meta. Reduz a aplicação a um protótipo que funcione ao menos em modo local. Lembra-te de que o objetivo principal é provar que consegues escrever código que resolve um problema concreto, não que a aplicação seja perfeita.

FAZ ISTO HOJE

Escolhe um objetivo concreto para a tua aplicação (por ex.: listar produtos do mercado).

Cria um repositório no GitHub e inicializa-o com um ``README.md``.

Define um cronograma de 90 dias dividido em blocos de duas semanas, anotando-o no teu diário de código.

Reserva três sessões de 45 minutos hoje: uma para teoria, outra para prática e a última para revisão.

Implementa um pequeno script que armazene três produtos num dicionário e mostre-os no terminal.

Conclusão

Quando te sentaste no candongueiro que atravessa o Bairro da Cacuaco, ao lado da Maria que vendia frutas no mercado, o sol ainda lançava sombras sobre as casas de chapa. O motor roncava, mas o teu pensamento já não estava ali; lembraste-te das linhas de código que tinhas escrito na primeira semana, dos erros que te fizeram rir e dos pequenos triunfos que fizeram o coração bater mais forte. Aquela viagem, que parecia só um deslocamento, transformou-se num reflexo da tua própria evolução: de quem não sabia distinguir um “if” de um “while” a quem já podia imaginar um aplicativo que ajudasse a tua comunidade a organizar o comércio ambulante.

Hoje, ao abrir a porta da tua casa e ver o ecrã iluminado, percebes que o caminho tem mais curvas do que esperavas, mas também que cada curva oferece uma nova oportunidade de aprender. Não há fórmula mágica que te garanta sucesso instantâneo, mas há a certeza de que a prática constante, a curiosidade e a vontade de ultrapassar os limites são as chaves que abrirão portas que ainda nem sabes que existem. Se alguma vez te sentires desanimado, lembra-te daquela primeira linha de código que escreveu, aquela que fez o teu programa dizer “Olá, mundo!” e sentei-me a pensar no que ainda poderás alcançar.

A jornada que começaste há três meses não termina aqui; é apenas o início de um caminho que podes trilhar ao ritmo que quiseses, com a mesma determinação que mostraste ao aprender a depurar aquele bug na noite de sexta-feira. Continua a experimentar, a partilhar o que

aprendes e a transformar ideias em realidade. O futuro está à tua espera, pronto para ser escrito em linhas de código que, como as histórias que contamos nas ruas de Luanda, deixarão marcas duradouras. Avança, porque o teu próximo projeto já está a chamar.

— *Osendre Plus*

FIM